WYDZIAŁ
MATEMATYKI
I FIZYKI STOSOWANEJ
POLITECHNIKI RZESZOWSKIEJ

Jakub Hutman i Krzysztof Koza

Projekt z przedmiotu Bazy Danych

Port morski (HUB)

Rzeszów, 2025

1. Projekt konceptualny 3
1.1. Sformułowanie zadania projektowego 3
1.2. Analiza stanu wyjściowego. 4
1.3. Wstępna analiza wymagań użytkownika 5
1.4. Scenariusze użycia 6
1.5. Identyfikacja funkcji systemu zarządzania portem morskim. 8
1.6. Propozycja encji i ich atrybutów oraz relacji 9
1.7. Diagram ERD 10
1.8. Tabela relacji 11
2. Projekt logiczny 12
2.1. Diagram relacyjny bazy danych 12
2.2. Opis bazy danych i atrybutów. 13
2.3. Normalizacja 26
2.4. Diagram relacyjny po normalizacji 39
2.5. Operacje na danych 40
3. Projekt implementacyjny 46
3.1. Rozwój aplikacji 46
3.2. Kod bazy danych 46
3.3. Wprowadzanie danych do bazy. 55
3.4. Kwerendy 65
4. Bibliografia 67

1. Projekt konceptualny.

1.1. Sformułowanie zadania projektowego

- Port morski pełni rolę huba transportowego, zajmując się obsługą i organizacją przesyłek cargo. Obsługa ta obejmuje przyjmowanie przesyłek, ich przeładunek, sortowanie, magazynowanie oraz organizację transportu wychodzącego – zarówno na inne statki, jak i ciężarówki. System ma na celu maksymalną optymalizację wykorzystania przestrzeni doków i magazynów, minimalizację przestojów oraz sprawną organizację przesyłek, aby efektywnie wspierać codzienną działalność logistyczną portu.

- Kluczowym wyzwaniem jest zarządzanie przestrzenią w dokach i magazynach, z jednoczesnym dbaniem o terminowość oraz dokładność obsługi przesyłek. Przesyłki rozdzielane są według typów, takich jak waga, gabaryty, czy rodzaj ładunku (np. Ładunki chłodnicze, przesyłki luzem, czy kontenery niebezpieczne). Dodatkowo system musi pozwalać na monitorowanie całego cyklu operacji – od wpływu statku, przez sortowanie przesyłek, po ich wysyłkę na kolejne środki transportu. Przesyłki mogą opuszczać port na innym statku bądź drogą lądową przy użyciu samochodów ciężarowych, w taki sposób, aby maksymalnie efektywnie zarządzać przestrzenią i czasem.

- Stworzymy system bazodanowy, który będzie wspierał pracowników portu i umożliwiał klientom dostęp do kluczowych informacji. System pozwala na rejestrację przesyłek, monitorowanie zajętości magazynów i doków, tworzenie harmonogramów przyjęć i wysyłek. Dzięki temu możliwe będzie śledzenie każdej przesyłki w czasie rzeczywistym oraz bieżące dostosowanie harmonogramów do dynamicznych zmian w logistyce portowej.

- Funkcje dla klientów: Klienci mogą zgłaszać przesyłki, podając ich kluczowe dane (waga, wymiary, rodzaj, preferowany termin wysyłki). Mają również możliwość śledzenia statusu przesyłek na każdym etapie procesu logistycznego – od przyjęcia, przez magazynowanie, po wysyłkę. System umożliwia im także wybór preferowanego środka transportu wychodzącego (statek lub samochód ciężarowy) oraz dostarcza powiadomienia o kluczowych etapach obsługi.

- Funkcje dla pracowników portu: Pracownicy mogą rejestrować nowe przesyłki, przypisywać je do doków i magazynów oraz zarządzać ich alokacją w oparciu o typ i wymagania ładunku. System pozwala na monitorowanie zajętości doków i magazynów w czasie rzeczywistym oraz sugeruje optymalne rozwiązania logistyczne. Pracownicy mogą także planować harmonogramy obsługi transportu przychodzącego i wychodzącego, przypisywać zasoby do konkretnych zadań.

1.2. Analiza stanu wyjściowego.

Port morski obecnie pełni rolę kluczowego huba transportowego, zajmującego się obsługą i organizacją przesyłek cargo. W obecnym stanie operacje portowe są częściowo zautomatyzowane, ale nadal opierają się w dużym stopniu na ręcznym zarządzaniu procesami logistycznymi. Brak centralnego systemu informatycznego skutkuje następującymi wyzwaniami:

- Nieefektywne zarządzanie przestrzenią:
  Magazyny i doki są zarządzane ręcznie lub za pomocą arkuszy kalkulacyjnych. Nie optymalnie wykorzystana przestrzeń powoduje przestoje i niską efektywność.

- Brak śledzenia przesyłek w czasie rzeczywistym:
  Przesyłki nie mają dedykowanego systemu monitorowania.Klienci często muszą kontaktować się bezpośrednio z pracownikami portu w celu uzyskania informacji o statusie przesyłek.

- Problemy z harmonogramowaniem:
  Brak centralnego systemu do planowania przyjęć i wysyłek statków oraz ciężarówek. Częste konflikty w alokacji doków i magazynów.

- Ograniczona kontrola nad procesami logistycznymi:
  Brak narzędzi analitycznych umożliwiających raportowanie i monitorowanie wydajności operacyjnej. Trudności w zarządzaniu zasobami, takimi jak personel, dźwigi i przestrzeń magazynowa.

- Ograniczona obsługa klientów:
  Klienci nie mają możliwości zgłaszania przesyłek ani monitorowania ich statusu online. Brak automatycznych powiadomień o kluczowych etapach obsługi przesyłek.

Podsumowując, obecny stan operacji portowych wymaga zintegrowanego systemu, który usprawni zarządzanie przestrzenią, harmonogramowanie oraz komunikację z klientami.

1.3. Wstępna analiza wymagań użytkownika

Wymagania klientów:

- Rejestracja przesyłek:
  - Możliwość wprowadzenia danych przesyłki, takich jak waga, wymiary, rodzaj ładunku i preferowany termin wysyłki.
  - Wybór środka transportu wychodzącego (statek lub samochód ciężarowy).

- Śledzenie przesyłek:
  - Dostęp do informacji o statusie przesyłki w czasie rzeczywistym.
  - Powiadomienia o kluczowych etapach procesu logistycznego (przyjęcie, przeładunek, wysyłka).

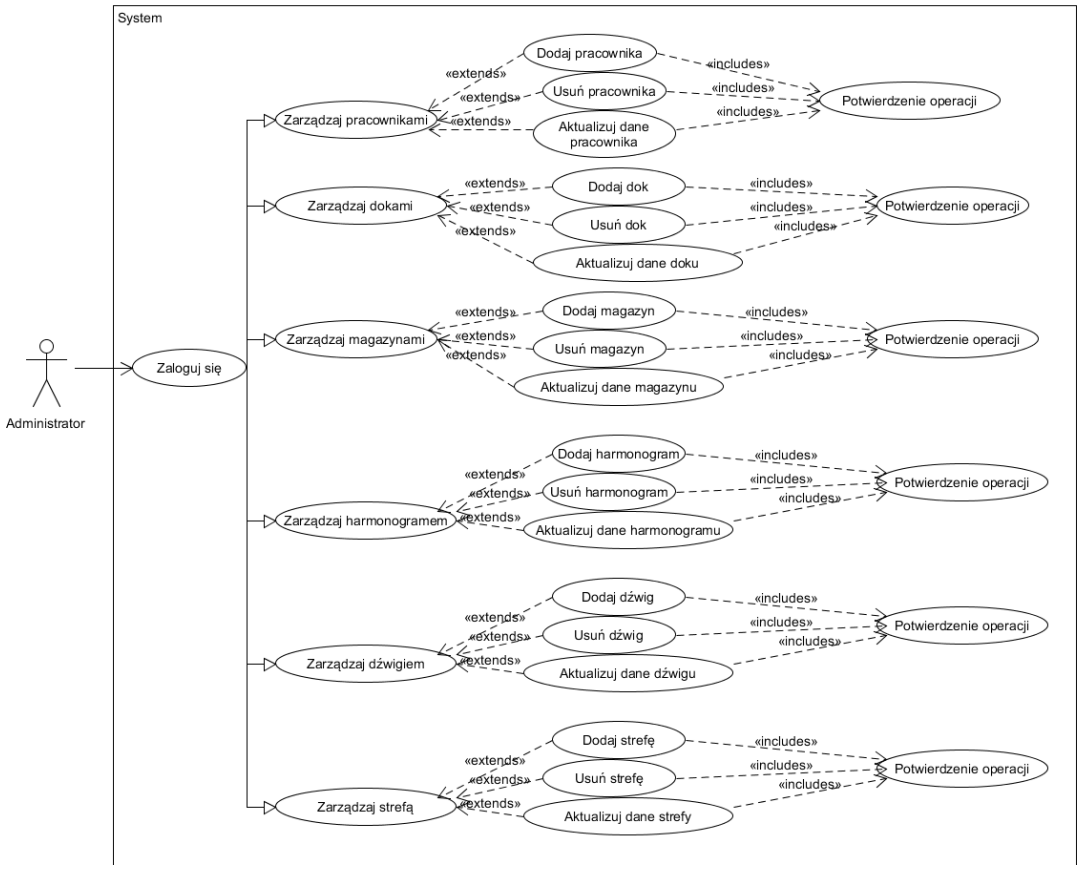
Wymagania pracowników portu:

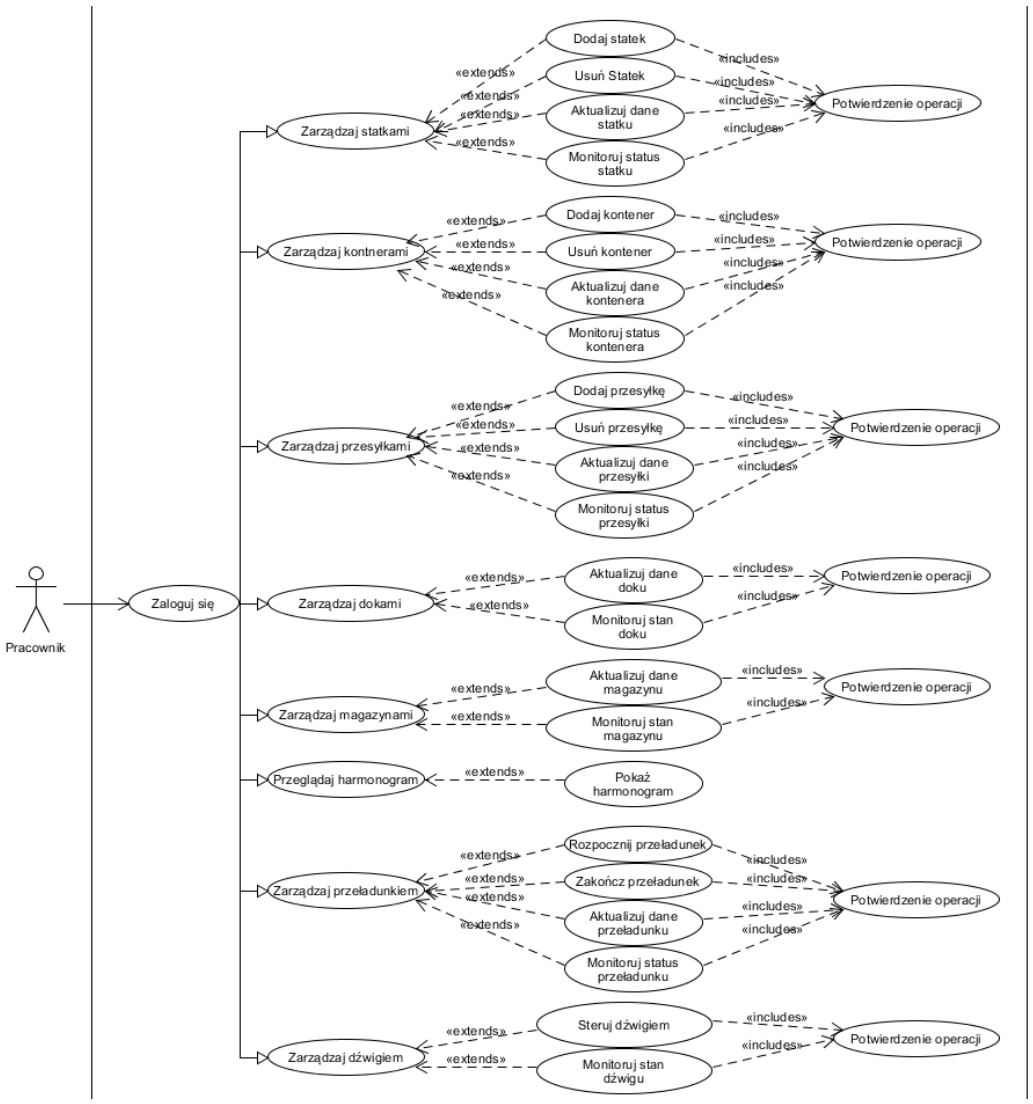
- Zarządzanie przestrzenią:
  - Monitorowanie zajętości doków i magazynów w czasie rzeczywistym.
  - Automatyczne sugestie dotyczące lokalizacji przesyłek na podstawie ich wymagań (waga, gabaryty, rodzaj ładunku).
- Rejestracja i obsługa przesyłek:
  - Szybka rejestracja nowych przesyłek i przypisywanie ich do odpowiednich zasobów (doki, magazyny).
  - Śledzenie całego cyklu operacji przesyłek.
- Harmonogramowanie:
  - Tworzenie i zarządzanie harmonogramami przyjęć statków, przeładunków i wysyłek.
  - Przypisywanie zasobów, takich jak personel i sprzęt, do konkretnych zadań.
- Monitorowanie i raportowanie:
  - Generowanie raportów dotyczących wydajności operacyjnej (przepustowość doków, czas przeładunków, zajętość magazynów).
  - Monitorowanie statusu operacji logistycznych w czasie rzeczywistym.

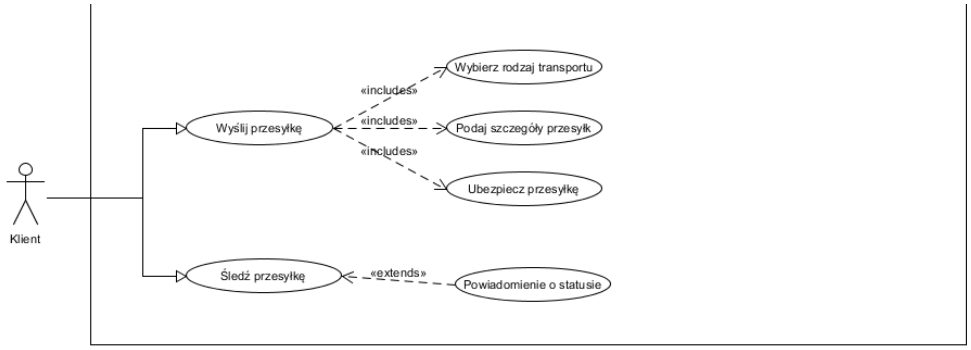
Wymagania systemowe:

- Zintegrowany system bazodanowy:
  - System musi przechowywać informacje o przesyłkach, statkach, magazynach, dokach i harmonogramach w sposób centralny.
- Interfejs dla klientów:
  - Prosty i intuicyjny interfejs umożliwiający rejestrację przesyłek, monitorowanie statusu i odbieranie powiadomień.
- Interfejs dla pracowników:
  - Rozbudowane narzędzia do zarządzania procesami logistycznymi i raportowania.
- Wysoka skalowalność i wydajność:
  - System musi być w stanie obsłużyć dużą ilość operacji i danych w czasie rzeczywistym.
- Bezpieczeństwo:
  - Ochrona danych klientów i przesyłek przed nieautoryzowanym dostępem.

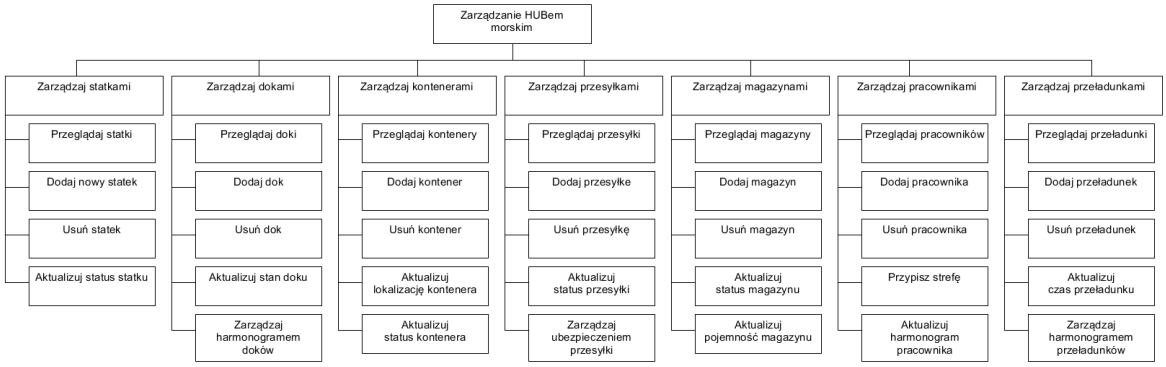
1.4. Scenariusze użycia




Pracownik
Zaloguj się
Zarządzaj statkami
Dodaj statek
Usuń Statek
Aktualizuj dane statku
Monitoruj status statku
Zarządzaj kontnerami
Dodaj kontener
Usuń kontener
Aktualizuj dane kontenera
Monitoruj status kontenera
Zarządzaj przesyłkami
Dodaj przesyłkę
Usuń przesyłkę
Aktualizuj dane przesyłki
Monitoruj status przesyłki
Zarządzaj dokami
Aktualizuj dane doku
Monitoruj stan doku
Zarządzaj magazynami
Aktualizuj dane magazynu
Monitoruj stan magazynu
Przeglądaj harmonogram
Pokaż harmonogram
Zarządzaj przeładunkiem
Rozpocznij przeładunek
Zakończ przeładunek
Aktualizuj dane przeładunku
Monitoruj status przeładunku
Zarządzaj dźwigiem
Steruj dźwigiem
Monitoruj stan dźwigu
Potwierdzenie operacji
«extends»
«includes»


Klient
Wyślij przesyłkę
Wybierz rodzaj transportu
Podaj szczegóły przesyłk
Ubezpiecz przesyłkę
Śledź przesyłkę
Powiadomienie o statusie
«includes»
«extends»

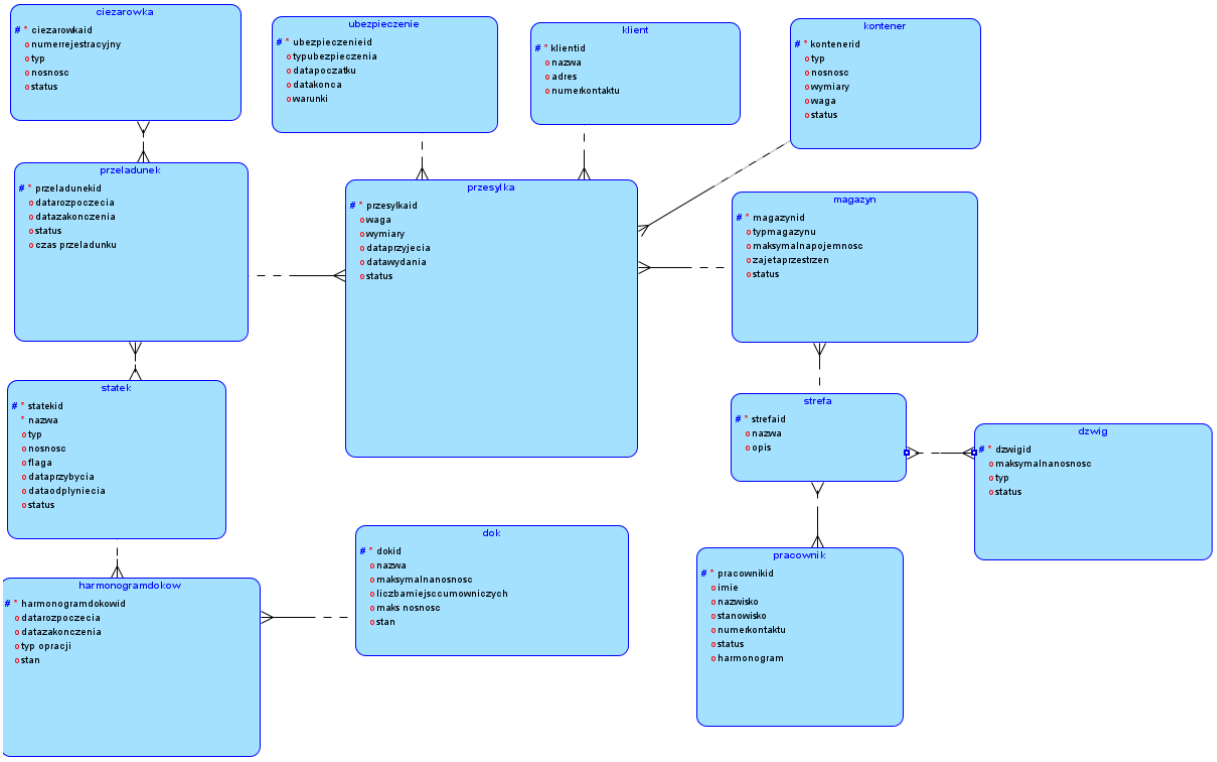
1.5. Identyfikacja funkcji systemu zarządzania portem morskim.



1.6. Propozycja encji i ich atrybutów oraz relacji

Encja	Atrybuty
CIEZAROWKA	ciezarowkaid (PK), numerrejestracyjny, typ, nosnosc, status
DOK	dokid (PK), nazwa, maksymalnanosnosc, liczbamiejsccumowniczych, stan
DZWIG	dzwigid (PK), strefaid (FK), maksymalnanosnosc, typ, status
KLIENT	klientid (PK), nazwa, adres, numerkontaktu
KONTENER	kontenerid (PK), typ, nosnosc, wymiary, waga, status
PRACOWNIK	pracownikid (PK), imie, nazwisko, stanowisko, numerkontaktu, strefaid (FK), status, harmonogram
PRZESYLKA	przesylkaid (PK), waga, wymiary, klientid (FK), magazynid (FK), dataprzyjecia, datawydania, status
STREFA	strefaid (PK), nazwa, opis
HARMONOGRAMDOKOW	harmonogramdokowid (PK), dokid (FK), datarozpoczecia, datazakonczenia, typ operacji, stan

1.7. Diagram ERD

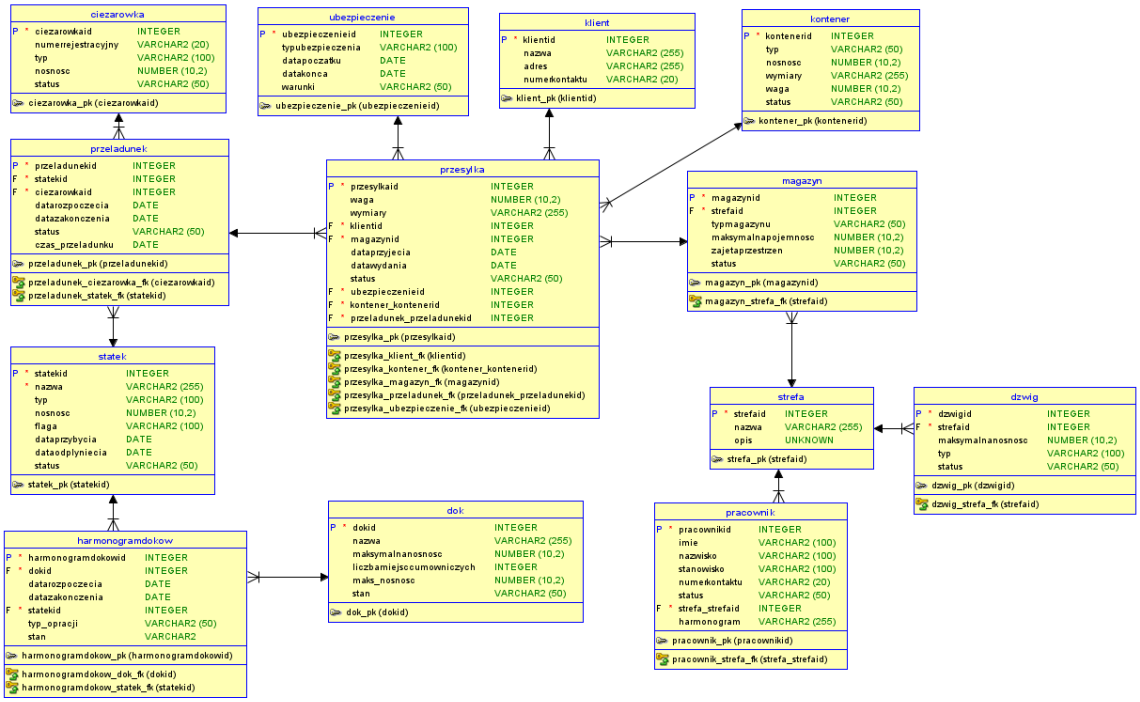


1.8. Tabela relacji

Nazwa relacji	Opis relacji
CIEZAROWKA-PRZELADUNEK	Relacja wiele do wielu. Wiele ciężarówek może być użyte w wielu przeładunkach.
PRZELADUNEK-STATEK	Relacja wiele do wielu. Jeden statek może uczestniczyć w wielu przeładunkach, a jeden przeładunek może dotyczyć wielu statków.
STATEK-HARMONOGRAMDOKOW	Relacja jeden do wielu. Jeden statek może mieć wiele wpisów w harmonogramie doków.
HARMONOGRAMDOKOW-DOK	Relacja wiele do jednego. Wiele harmonogramów może być powiązanych z jednym dokiem.
PRZESYLKA-KLIENT	Relacja wiele do jednego. Wiele przesyłek może należeć do jednego klienta.
PRZESYLKA-KONTENER	Relacja wiele do jednego. Jeden kontener może zawierać wiele przesyłek.
PRZESYLKA-MAGAZYN	Relacja wiele do jednego. Wiele przesyłek może być przechowywanych w jednym magazynie.
DZWIG-STREFA	Relacja wiele do wielu. Jeden dźwig może obsługiwać wiele stref, a jedna strefa może mieć wiele dźwigów.
PRACOWNIK-STREFA	Relacja wiele do wielu. Jeden pracownik może być przypisany do wielu stref, a jedna strefa może mieć wielu pracowników.
MAGAZYN-STREFA	Relacja wiele do jednego. Wiele magazynów może znajdować się w jednej strefie.
PRZESYLKA-UBEZPIECZENIE	Relacja wiele do jednego. Wiele przesyłek może być objętych jednym typem ubezpieczenia.

2. Projekt logiczny

2.1. Diagram relacyjny bazy danych



2.2. Opis bazy danych i atrybutów.

- Tabela: CIEZAROWKA
  Przedstawia ciężarówki wykorzystywane w systemie, np. do transportu.

  Atrybuty:
  - ciezarowkaid
    Każda ciężarówka potrzebuje unikalnego identyfikatora, aby można było ją odróżnić od innych.
    Typ danych: INTEGER

  - numerrejestracyjny
    Każda ciężarówka ma swój unikalny numer rejestracyjny, który pozwala ją zidentyfikować w świecie rzeczywistym.
    Typ danych: VARCHAR2(20)
    Długość: 20 znaków wystarczy, aby przechować dowolny numer rejestracyjny.

  - **typ**
    Wskazuje rodzaj ciężarówki, np. naczepa, chłodnia itp.
    Typ danych: VARCHAR2(100)
    Długość: 100 znaków to zapas na opis nawet bardzo długiej nazwy typu ciężarówki.

  - nosnosc
    Przechowuje maksymalną wagę, jaką ciężarówka może przewieźć.
    Typ danych: NUMBER(10, 2)
    Długość: 10 cyfr z czego 2 to miejsca po przecinku, aby przechować precyzyjnie nośność nawet bardzo dużych ciężarówek.

  - **status**
    Wskazuje, czy ciężarówka jest dostępna, w naprawie, czy używana.
    Typ danych: VARCHAR2(50)
    Długość: 50 znaków to wystarczająco dużo na przechowanie opisu statusu.

- Tabela: DOK

Przedstawia doki przeładunkowe, gdzie obsługiwane są towary.

Atrybuty:

- dokid
  Każdy dok musi mieć unikalny identyfikator, aby można go było jednoznacznie zidentyfikować.
  Typ danych: INTEGER

- nazwa
  Pozwala nadać dokowi nazwę, np. "Dok 1", "Dok dla kontenerów".
  Typ danych: VARCHAR2(255)
  Długość: 255 znaków odpowiednie dla dlugich nazw.

- maksymalnanosnosc
  Określa maksymalną wagę, jaką dok może obsłużyć.
  Typ danych: NUMBER(10, 2)
  Długość: 10 cyfr z 2 miejscami po przecinku wystarczy na opisanie nawet bardzo dużych wartości.

- liczbamiejsccumowniczych
  Przechowuje liczbę miejsc dla cumowania statków w doku.
  Typ danych: INTEGER
  Liczba całkowita jest wystarczająca, ponieważ liczba miejsc jest zawsze liczbą całkowitą.

- **stan**
  Określa aktualny stan doku, np. "Dostępny", "W naprawie".
  Typ danych: VARCHAR2(50)
  Długość: 50 znaków to wystarczający zapas na różne opisy stanu.

- Tabela: DZWIG

Przedstawia dźwigi obsługujące towary w porcie.

Atrybuty:

- dzwigid
  Każdy dźwig ma unikalny identyfikator, aby go odróżnić.
  Typ danych: INTEGER

- strefaid
  Wskazuje, w której strefie dźwig się znajduje (relacja ze strefą).
  Typ danych: INTEGER

- maksymalnanosnosc
  Wskazuje maksymalny ciężar, jaki dźwig może unieść.
  Typ danych: NUMBER(10, 2)
  Długość: 10 cyfr, w tym 2 miejsca po przecinku.

- typ
  Określa rodzaj dźwigu, np. "Hydrauliczny", "Elektryczny".
  Typ danych: VARCHAR2(100)
  Długość: 100 znaków zapewnia wystarczająco miejsca na różnorodne nazwy.

- status
  Określa, czy dźwig jest dostępny do pracy.
  Typ danych: VARCHAR2(50)
  Długość: 50 znaków to wystarczająca ilość.

- Tabela: HARMONOGRAMDOKOW
  Przechowuje informacje o harmonogramach użycia doków.

  Atrybuty:
  - harmonogramdokowid
    Potrzebujemy unikalnego identyfikatora dla każdego harmonogramu, aby jednoznacznie rozróżnić wpisy.
    Typ danych: INTEGER

  - dokid
    Wskazuje, który dok dotyczy danego harmonogramu.
    Typ danych: INTEGER
    Liczba całkowita odnosi się do klucza głównego w tabeli dok.

  - datarozpoczecia
    Określa, kiedy harmonogram dla doku zaczyna obowiązywać.
    Typ danych: DATE
    Wybrano typ DATE, ponieważ precyzyjnie przechowuje daty.

  - datazakonczenia
    Określa, kiedy harmonogram dla doku się kończy.
    Typ danych: DATE
    Typ daty pozwala dokładnie przechowywać informacje o zakończeniu.

  - statekid
    Wskazuje, który statek jest powiązany z danym harmonogramem.
    Typ danych: INTEGER
    Liczba całkowita odwołuje się do klucza głównego w tabeli statek.

  - **typ_opracji**
    Określa rodzaj operacji zaplanowanej dla doku, np. załadunek lub rozładunek.
    Typ danych: VARCHAR2(50)
    Długość: 50 znaków wystarczy na opis dowolnej operacji.

  - stan
    Określa status harmonogramu, np. "Zatwierdzony", "W trakcie".
    Typ danych: VARCHAR2(50)
    Długość: 50 znaków wystarczy na różne statusy.

- Tabela: KLIENT

Reprezentuje klientów korzystających z usług.

Atrybuty:

- klientid
  Każdy klient ma unikalny identyfikator dla jednoznacznej identyfikacji.
  Typ danych: INTEGER

- nazwa
  Przechowuje nazwę klienta, np. nazwę firmy.
  Typ danych: VARCHAR2(255)
  Długość: 255 znaków wystarczy na nawet bardzo długie nazwy.

- adres
  Przechowuje adres klienta, przyjeliśmy, że jest to miasto.
  Typ danych: VARCHAR2(255).
  Długość: 255 znaków pozwala na przechowanie nawet długiego adresu.

- numerkontaktu
  Zawiera numer telefonu lub kontaktowy klienta.
  Typ danych: VARCHAR2(20)
  Długość: 20 znaków to wystarczający zakres dla numerów telefonów.

- Tabela: PRZESYLKA
  Reprezentuje przesyłki obsługiwane przez port.

  Atrybuty:
  - przesylkaid
    Jest to unikalny identyfikator każdej przesyłki. Dzięki temu można jednoznacznie odróżnić jedną przesyłkę od innej w systemie.
    Typ danych: INTEGER

  - waga
    Przechowuje wagę przesyłki, co jest kluczowe dla logistyki, wyceny i planowania transportu.
    Typ danych: NUMBER(10, 2)
    Długość: 10 cyfr, w tym 2 miejsca po przecinku, zapewnia możliwość zapisu nawet bardzo dużych wag.

  - wymiary
    Określa wymiary przesyłki (np. długość, szerokość i wysokość), co jest istotne przy planowaniu transportu i przeładunku.
    Typ danych: VARCHAR2(255), ponieważ wymiary mogą być zapisywane w różny sposób, np. "200x150x100 cm".
    Długość: 255 znaków to zapas na opis dowolnych wymiarów.

  - klientid
    Każda przesyłka musi być powiązana z klientem, który ją zleca, aby wiedzieć, do kogo należy.
    Typ danych: INTEGER

  - magazynid
    Wskazuje, w którym magazynie znajduje się dana przesyłka, co umożliwia jej śledzenie w systemie.
    Typ danych: INTEGER

- dataprzyjecia
  Przechowuje datę, kiedy przesyłka została przyjęta do magazynu, co jest istotne dla logistyki i rozliczeń.
  Typ danych: DATE

- datawydania
  Określa datę, kiedy przesyłka została wydana z magazynu, co jest kluczowe dla śledzenia czasu przechowywania.
  Typ danych: DATE

- status
  Przechowuje status przesyłki, np. "W magazynie", "Wysłana", co pomaga w śledzeniu przesyłki w systemie.
  Typ danych: VARCHAR2(50)
  Długość: 50 znaków wystarcza na różnorodne statusy.

- ubezpieczenieid
  Określa, jakie ubezpieczenie jest związane z przesyłką, jeśli takie istnieje.
  Typ danych: INTEGER

- kontener**_kontenerid**
  Wskazuje, w którym kontenerze znajduje się przesyłka, co ułatwia jej lokalizację.
  Typ danych: INTEGER

- przeladunek**_przeladunekid**
  Określa, w ramach jakiego przeładunku przesyłka została przeniesiona.
  Typ danych: INTEGER

- Tabela: KONTENER
  Reprezentuje kontenery, w których przewożone są towary.

  Atrybuty:
  - kontenerid
    Służy jako unikalny identyfikator dla każdego kontenera, aby można je było jednoznacznie rozróżnić w systemie.
    Typ danych: INTEGER

  - typ
    Określa rodzaj kontenera, np. "Standardowy", "Chłodniczy".
    Typ danych: VARCHAR2(50)
    Długość: 50 znaków wystarczy na różne typy.

  - nosnosc
    Przechowuje maksymalną wagę, jaką kontener może pomieścić, co jest istotne przy planowaniu załadunku.
    Typ danych: NUMBER(10, 2)
    Długość: 10 cyfr, w tym 2 miejsca po przecinku.

  - wymiary
    Określa wymiary kontenera, co jest istotne przy planowaniu miejsca na statku lub w magazynie.
    Typ danych: VARCHAR2(255)
    Tekst pozwala na elastyczne zapisanie wymiarów w dowolnym formacie, np. "6m x 2.5m x 2.5m".
    Długość: 255 znaków to wystarczający zapas.

  - waga
    Przechowuje wagę pustego kontenera, co jest istotne przy obliczaniu całkowitej wagi ładunku.
    Typ danych: NUMBER(10, 2).
    Długość: 10 cyfr, w tym 2 miejsca po przecinku.

  - status
    Informuje o stanie kontenera, np. "W użyciu", "Pusty".
    Typ danych: VARCHAR2(50)
    Długość: 50 znaków wystarczy na różnorodne opisy.

- Tabela: MAGAZYN
  Reprezentuje magazyny używane do przechowywania towarów.

  Atrybuty:
  - magazynid
    Unikalny identyfikator dla każdego magazynu, aby można je było odróżnić.
    Typ danych: INTEGER

  - strefaid
    Wskazuje, w której strefie znajduje się magazyn.
    Typ danych: INTEGER

  - typmagazynu
    Określa typ magazynu, np. "Chłodnia", "Standardowy".
    Typ danych: VARCHAR2(50)
    Długość: 50 znaków to wystarczający zapas.

  - maksymalnapojemnosc
    Określa maksymalną pojemność magazynu, co jest istotne przy planowaniu przechowywania towarów.
    Typ danych: NUMBER(10, 2)
    Długość: 10 cyfr, w tym 2 miejsca po przecinku.

  - zajetaprzestrzen
    Informuje, ile miejsca w magazynie jest już zajęte.
    Typ danych: NUMBER(10, 2)
    Długość: 10 cyfr, w tym 2 miejsca po przecinku.

  - status
    Określa status magazynu, np. "Pełny", "Wolny".
    Typ danych: VARCHAR2(50)
    Długość: 50 znaków wystarcza na różne opisy.

- Tabela: PRACOWNIK
  Reprezentuje pracowników zatrudnionych w porcie.

  Atrybuty:
  - pracownikid
    Unikalny identyfikator dla każdego pracownika, aby jednoznacznie ich identyfikować.
    Typ danych: INTEGER

  - imie
    Przechowuje imię pracownika.
    Typ danych: VARCHAR2(100)
    Długość: 100 znaków daje zapas na dłuższe imiona.

  - nazwisko
    Przechowuje nazwisko pracownika.
    Typ danych: VARCHAR2(100)
    Długość: 100 znaków wystarczy.

  - stanowisko
    Określa stanowisko pracownika, np. "Kierowca", "Operator dźwigu".
    Typ danych: VARCHAR2(100)
    Długość: 100 znaków daje zapas.

  - numerkontaktu
    Przechowuje numer telefonu pracownika.
    Typ danych: VARCHAR2(20)
    Długość: 20 znaków wystarcza.

  - status
    Określa status pracownika, np. "Zatrudniony", "Na urlopie".
    Typ danych: VARCHAR2(50)
    Długość: 50 znaków wystarcza.

  - strefa_strefaid
    Wskazuje, w której strefie pracuje dany pracownik.
    Typ danych: INTEGER

  - harmonogram
    Przechowuje informacje o harmonogramie pracy pracownika.
    Typ danych: VARCHAR2(255)
    Długość: 255 znaków to wystarczający zapas.

- Tabela: PRZELADUNEK
  Reprezentuje operacje przeładunku towarów między różnymi środkami transportu.

  Atrybuty:
  - przeladunekid
    Jest to unikalny identyfikator przeładunku, który pozwala odróżnić różne operacje w systemie.

  - statekid
    Wskazuje statek związany z przeładunkiem, co umożliwia powiązanie operacji ze środkiem transportu.
    Typ danych: INTEGER

  - ciezarowkaid
    Określa ciężarówkę zaangażowaną w przeładunek.
    Typ danych: INTEGER

  - datarozpoczecia
    Rejestruje datę rozpoczęcia operacji przeładunku.
    Typ danych: DATE

  - datazakonczenia
    Rejestruje datę zakończenia przeładunku.
    Typ danych: DATE

  - status
    Określa aktualny status przeładunku, np. "W trakcie", "Zakończony".
    Typ danych: VARCHAR2(50)
    Długość: 50 znaków wystarczy na różne opisy.

  - czas_przeladunku
    Określa łączny czas przeładunku, co może być użyteczne w analizach wydajności.
    Typ danych: DATE

- Tabela: STATEK
  Reprezentuje statki korzystające z usług portu.

  Atrybuty:
  - statekid
    Unikalny identyfikator statku, który pozwala jednoznacznie identyfikować każdy statek w systemie.
    Typ danych: INTEGER

  - nazwa
    Przechowuje nazwę statku, np. "Horyzont II".
    Typ danych: VARCHAR2(255)
    Długość: 255 znaków to zapas na długie nazwy.

  - typ
    Określa typ statku, np. "Kontenerowiec", "Tankowiec".
    Typ danych: VARCHAR2(100)
    Długość: 100 znaków daje zapas.

  - nosnosc
    Przechowuje maksymalną nośność statku, co jest istotne dla planowania ładunków.
    Typ danych: NUMBER(10, 2)
    Długość: 10 cyfr, w tym 2 miejsca po przecinku.

  - flaga
    Przechowuje informację o kraju rejestracji statku.
    Typ danych: VARCHAR2(100)
    Długość: 100 znaków wystarczy na dowolną nazwę kraju.

  - dataprzybycia
    Rejestruje datę przybycia statku do portu.
    Typ danych: DATE

  - dataodplyniecia
    Rejestruje datę, kiedy statek opuścił port.
    Typ danych: DATE

  - status
    Określa aktualny status statku, np. "Oczekuje", "Załadowany".
    Typ danych: VARCHAR2(50)
    Długość: 50 znaków wystarcza.

- Tabela: STREFA
  Reprezentuje różne strefy w porcie, np. miejsca magazynowania lub przeładunku.

  Atrybuty:
  - strefaid
    Unikalny identyfikator dla każdej strefy.
    Typ danych: INTEGER

  - nazwa
    Przechowuje nazwę strefy, np. "Strefa 1".
    Typ danych: VARCHAR2(255)
    Długość: 255 znaków wystarcza.

  - opis
    Przechowuje dodatkowe informacje o strefie.
    Typ danych: VARCHAR2(255)
    Długość: 255 znaków wystarcza.

- Tabela: UBEZPIECZENIE
  Reprezentuje ubezpieczenia przypisane do przesyłek.

  Atrybuty:
  - ubezpieczenieid
    Unikalny identyfikator dla każdego ubezpieczenia.
    Typ danych: INTEGER

  - typubezpieczenia
    Przechowuje rodzaj ubezpieczenia, np. "Pełne", "Ograniczone".
    Typ danych: VARCHAR2(100)
    Długość: 100 znaków wystarczy.

  - datapoczatku
    Przechowuje datę rozpoczęcia ubezpieczenia.
    Typ danych: DATE

  - datakonca
    Przechowuje datę zakończenia ubezpieczenia.
    Typ danych: DATE

  - warunki
    Przechowuje warunki ubezpieczenia w formie opisu.
    Typ danych: VARCHAR2(50)
    Długość: 50 znaków wystarcza.

2.3. Normalizacja

- **Tabela dok**

Przed normalizacją:

Problem: Informacje o nośności i stanie doku nie były wystarczająco szczegółowe
Po normalizacji:

Tabela dok
Atrybuty:
- dokid (INTEGER): Klucz główny identyfikujący unikalnie każdy dok.
- nazwa (VARCHAR2(255)): Nazwa doku.
- maks_nosnosc (NUMBER(10, 2)): Maksymalna nośność doku w tonach.

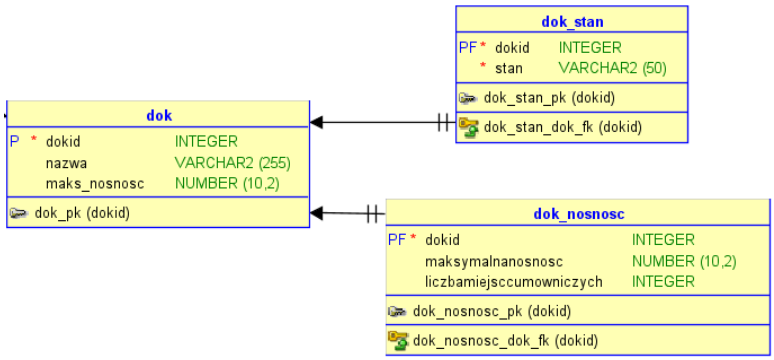
Powód istnienia:
Przechowuje podstawowe dane o dokach, które są niezależne od innych cech

Tabela dok_nosnosc
Atrybuty:
- dokid (INTEGER): Klucz obcy wskazujący na dok.
- maksymalnanosnosc (NUMBER(10, 2)): Szczegółowa nośność doku.
- liczbamiejsccumowniczych (INTEGER): Liczba miejsc cumowniczych dostępnych w doku.

Powód istnienia:
Dane dotyczące nośności są specyficzne dla każdego doku i mogą być używane osobno

Tabela dok_stan
Atrybuty:
- dokid (INTEGER): Klucz obcy wskazujący na dok.
- stan (VARCHAR2(50)): Aktualny stan doku (np. „aktywny”, „w naprawie”).

Powód istnienia:
Stan doku zmienia się dynamicznie i jest niezależny od podstawowych danych o doku.



- **Tabela harmonogramdokow**

Przed normalizacją:

Problem: Dane takie jak typ_opracji i stan są zależne od innych danych.

Po normalizacji:

Tabela harmonogramdokow
Atrybuty:
- harmonogramdokowid (INTEGER): Klucz główny identyfikujący unikalnie harmonogram.
- dokid (INTEGER): Klucz obcy wskazujący na dok.
- datarozpoczecia (DATE): Data rozpoczęcia operacji w doku.
- datazakonczenia (DATE): Data zakończenia operacji w doku.
- statekid (INTEGER): Klucz obcy wskazujący na statek.

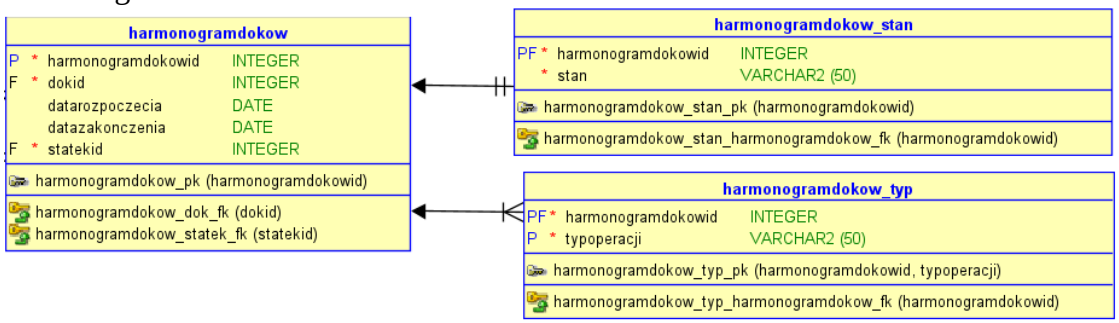
Powód istnienia:
Przechowuje podstawowe dane o harmonogramie operacji w doku.

Tabela harmonogramdokow_stan
Atrybuty:
- harmonogramdokowid (INTEGER): Klucz obcy wskazujący na harmonogramdokow.
- stan (VARCHAR2(50)): Stan operacji (np. „w trakcie”, „zakończony”).

Powód istnienia:
Stan operacji może zmieniać się dynamicznie i jest niezależny od danych o harmonogramie.

Tabela harmonogramdokow_typ
Atrybuty:
- harmonogramdokowid (INTEGER): Klucz obcy wskazujący na harmonogramdokow.
- typoperacji (VARCHAR2(50)): Typ operacji wykonywanej w harmonogramie (np. „rozładunek”, „załadunek”).

Powód istnienia:
Typ operacji pozwala na dokładne określenie rodzaju czynności związanej z harmonogramem.



- ## Tabela kontener

Przed normalizacją:

Problem: Dane o statusie i lokalizacji były często zależne od innych atrybutów.

Po normalizacji:

Tabela kontener
Atrybuty:
- kontenerid (INTEGER): Klucz główny.
- typ (VARCHAR2(50)): Typ kontenera (np. „chłodniczy”, „standardowy”).
- nosnosc (NUMBER(10, 2)): Maksymalna nośność kontenera.
- typkontenera (VARCHAR2(100)): Szczegółowy typ kontenera.
- wymiary (VARCHAR2(255)): Wymiary kontenera.
- waga (NUMBER(10, 2)): Waga pustego kontenera.

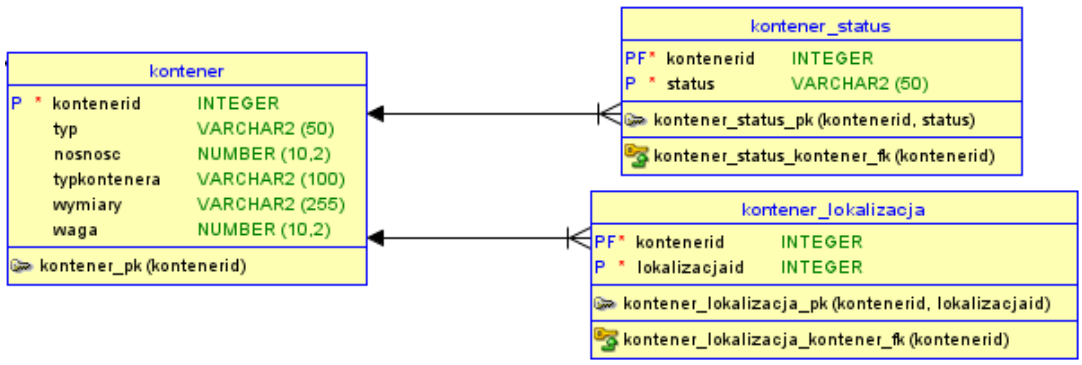
Powód istnienia:
Przechowuje dane statyczne o kontenerze, które nie zmieniają się dynamicznie.

Tabela kontener_lokalizacja
Atrybuty:
- kontenerid (INTEGER): Klucz obcy wskazujący na kontener.
- lokalizacjaid (INTEGER): Identyfikator lokalizacji kontenera.

Powód istnienia:
Lokalizacja kontenera zmienia się dynamicznie, więc została oddzielona od danych statycznych.

Tabela kontener_status
Atrybuty:
- kontenerid (INTEGER): Klucz obcy wskazujący na kontener.
- status (VARCHAR2(50)): Status kontenera (np. „wolny”, „używany”).

Powód istnienia:
Status kontenera może się zmieniać dynamicznie, dlatego został wydzielony.



- ## Tabela przesylka

Przed normalizacją:

Problem: Dane dotyczące kontenera i typu przesyłki nie były w pełni zależne od klucza głównego.

Po normalizacji:

Tabela przesylka
Atrybuty:
- przesylkaid (INTEGER): Klucz główny.
- waga (NUMBER(10, 2)): Waga przesyłki.
- wymiary (VARCHAR2(255)): Wymiary przesyłki.
- klientid (INTEGER): Klucz obcy wskazujący na klienta.
- magazynid (INTEGER): Klucz obcy wskazujący na magazyn.
- dataprzyjecia (DATE): Data przyjęcia przesyłki.
- datawydania (DATE): Data wydania przesyłki.
- status (VARCHAR2(50)): Status przesyłki.
- typprzesylkiid (INTEGER): Klucz obcy wskazujący na typ przesyłki.
- ubezpieczenieid (INTEGER): Klucz obcy wskazujący na ubezpieczenie.

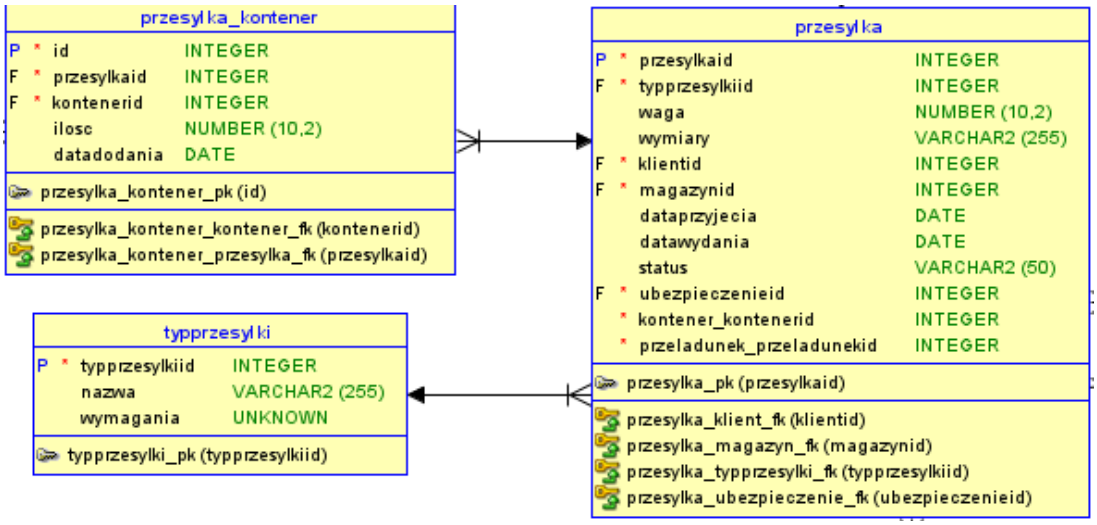
Powód istnienia:
Przechowuje dane o przesyłkach, które są podstawowe i statyczne.

Tabela przesylka_kontener
Atrybuty:
- id (INTEGER): Klucz główny.
- przesylkaid (INTEGER): Klucz obcy wskazujący na przesylka.
- kontenerid (INTEGER): Klucz obcy wskazujący na kontener.
- ilosc (NUMBER(10, 2)): Ilość przesyłek w kontenerze.
- datadodania (DATE): Data dodania przesyłki do kontenera.

Powód istnienia:
Relacja między przesyłką a kontenerem jest dynamiczna i wymaga dedykowanej encji

Tabela typprzesylki
Atrybuty:
- typprzesylkiid (INTEGER): Klucz główny.
- nazwa (VARCHAR2(255)): Nazwa typu przesyłki (np. „żywność”, „chemikalia”).
- wymagania (VARCHAR(255)): Szczegółowe wymagania dotyczące przesyłki.

Powód istnienia:
Ułatwia zarządzanie różnymi typami przesyłek.



- ## Tabela przeladunek

Przed normalizacją:

Problem: Dane o czasie przeładunku oraz związanych z nim przesyłkach były przechowywane w jednym miejscu, co powodowało nadmiarowosc.

Po normalizacji:

Tabela przeladunek
Atrybuty:
- przeladunekid (INTEGER): Klucz główny identyfikujący przeładunek.
- statekid (INTEGER): Klucz obcy wskazujący na statek uczestniczący w przeładunku.
- ciezarowkaid (INTEGER): Klucz obcy wskazujący na ciężarówkę uczestniczącą w przeładunku.
- status (VARCHAR2(50)): Status przeładunku (np. „w trakcie”, „zakończony”).
- harmonogramprzeladunkow_przeladunekid (INTEGER): Klucz obcy wskazujący na harmonogram przeładunków.
- przeladunek_czas_przeladunekid (INTEGER): Klucz obcy wskazujący na tabelę z czasem przeładunku.

Powód istnienia:
Przechowuje podstawowe dane o przeładunkach, które są kluczowe dla ich identyfikacji.

Tabela przeladunek_czas
Atrybuty:

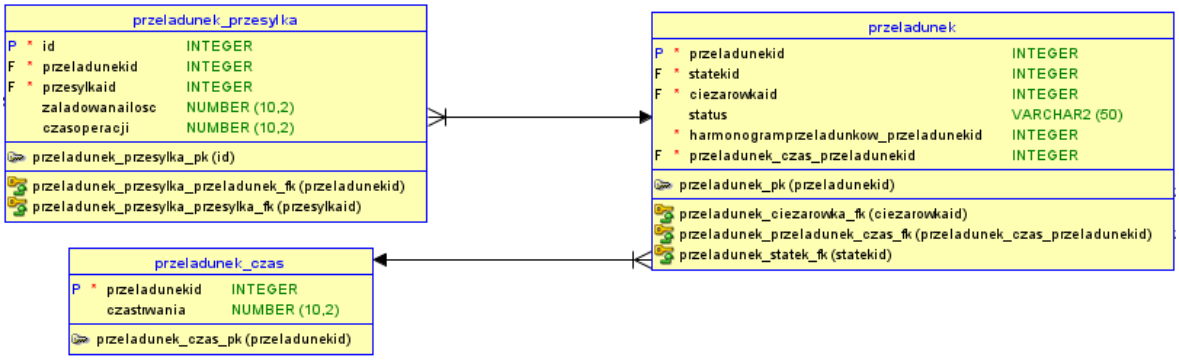
- przeladunekid (INTEGER): Klucz obcy wskazujący na przeładunek.
- czastrwania (NUMBER(10, 2)): Czas trwania przeładunku w godzinach.

Powód istnienia:
Dane o czasie przeładunku zostały oddzielone, aby umożliwić bardziej precyzyjne zarządzanie i analizę czasów operacji.

Tabela przeladunek_przesylka
Atrybuty:

- id (INTEGER): Klucz główny.
- przeladunekid (INTEGER): Klucz obcy wskazujący na przeładunek.
- przesylkaid (INTEGER): Klucz obcy wskazujący na przesyłkę.
- zaladowanailosc (NUMBER(10, 2)): Ilość przesyłek załadowanych podczas operacji.
- czasoperacji (NUMBER(10, 2)): Czas trwania operacji związanej z konkretną przesyłką.

Powód istnienia:
Rozdziela szczegóły dotyczące przesyłek obsługiwanych w ramach przeładunku.



- ## Tabela ubezpieczenie

Przed normalizacją:

Problem: Dane o typach ubezpieczenia i ich warunkach były zagnieżdżone w jednej tabeli.

Po normalizacji:

Tabela ubezpieczenie
Atrybuty:
- ubezpieczenieid (INTEGER): Klucz główny identyfikujący ubezpieczenie.
- datapoczatku (DATE): Data rozpoczęcia ubezpieczenia.
- datakonca (DATE): Data zakończenia ubezpieczenia.

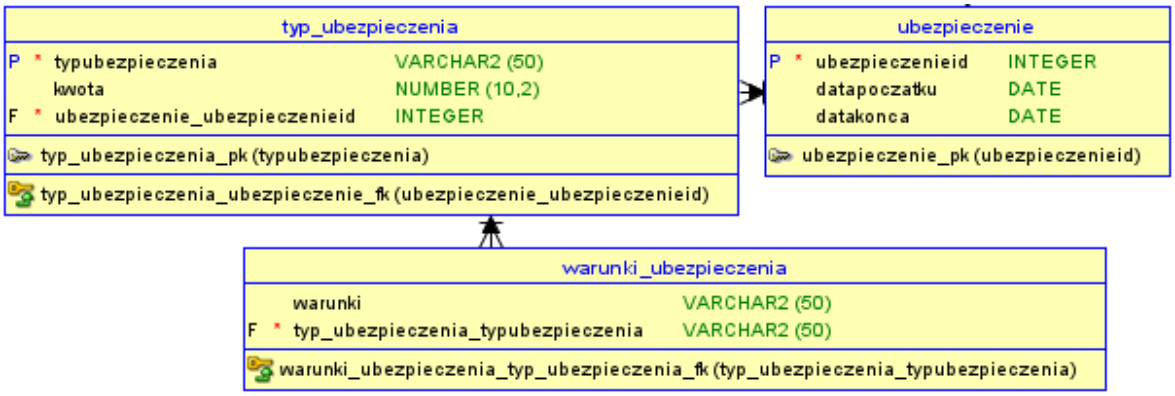
Powód istnienia:
Przechowuje dane o konkretnych polisach ubezpieczeniowych.

Tabela typ_ubezpieczenia
Atrybuty:
- typubezpieczenia (VARCHAR2(50)): Nazwa typu ubezpieczenia (np. „transportowe", „szkody mechaniczne").
- kwota (NUMBER(10, 2)): Koszt ubezpieczenia.
- ubezpieczenie_ubezpieczenieid (INTEGER): Klucz obcy wskazujący na ubezpieczenie.

Powód istnienia:
Oddziela typy ubezpieczeń i ich koszty, co pozwala na elastyczniejsze zarządzanie nimi.

Tabela warunki_ubezpieczenia
Atrybuty:
- warunki (VARCHAR2(50)): Opis warunków ubezpieczenia (np. „obejmuje kradzież").
- typ_ubezpieczenia_typubezpieczenia (VARCHAR2(50)): Klucz obcy wskazujący na typ_ubezpieczenia.

Powód istnienia:
Przechowuje szczegóły warunków dla każdego typu ubezpieczenia.



- ## Tabela pracownik

Przed normalizacją:

Problem: Dane o przypisanych strefach i harmonogramie były przechowywane w jednym miejscu.

Po normalizacji:

Nie udało się osiągnąć wymaganej normalizacji tabeli. Normalizację zakończyliśmy na 2NF, stanowisko pracownika jest wielowartościowe, jest to błąd którego jesteśmy świadomi.

Tabela pracownik
Atrybuty:
- pracownikid (INTEGER): Klucz główny identyfikujący pracownika.
- imie (VARCHAR2(100)): Imię pracownika.
- nazwisko (VARCHAR2(100)): Nazwisko pracownika.
- stanowisko (VARCHAR2(100)): Stanowisko zajmowane przez pracownika(wielowartościowa).
- numerkontaktu (VARCHAR2(20)): Numer kontaktowy pracownika.
- status (VARCHAR2(50)): Status pracownika (np. „aktywny”).

Powód istnienia:
Przechowuje podstawowe dane o pracownikach.

Tabela pracownik_strefa
Atrybuty:
- id (INTEGER): Klucz główny.
- pracownikid (INTEGER): Klucz obcy wskazujący na pracownik.
- strefaid (INTEGER): Klucz obcy wskazujący na strefa.

Powód istnienia:
Relacja między pracownikami a strefami została oddzielona, aby umożliwić zarządzanie przypisaniami dynamicznie.

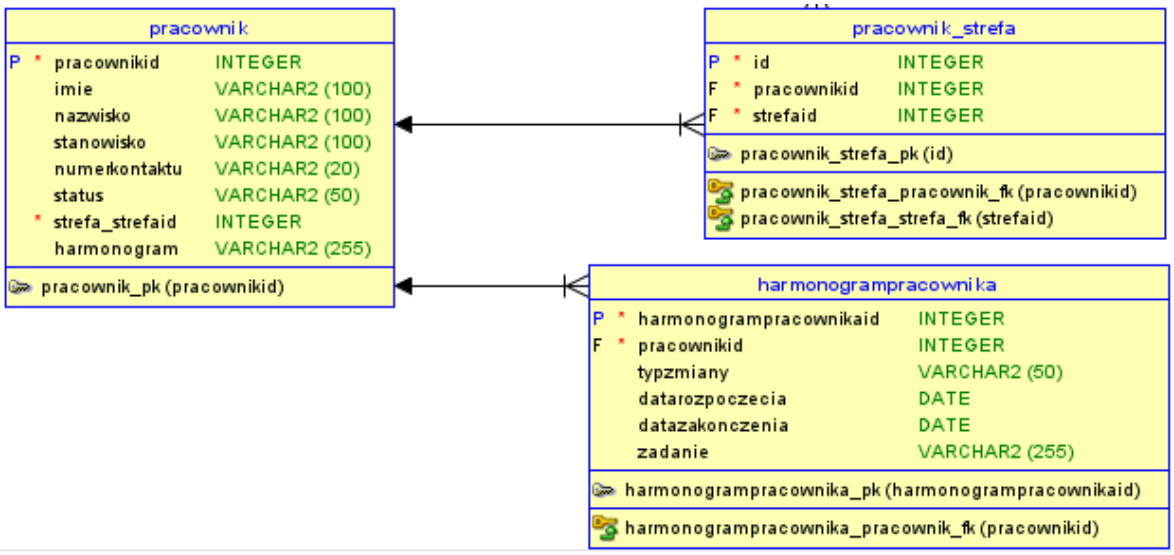
Tabela harmonogrampracownika

Atrybuty:

- harmonogrampracownikaid (INTEGER): Klucz główny.
- pracownikid (INTEGER): Klucz obcy wskazujący na pracownik.
- typzmiany (VARCHAR2(50)): Typ zmiany pracownika (np. „ranna”, „nocna”).
- datarozpoczecia (DATE): Data rozpoczęcia zmiany.
- datazakonczenia (DATE): Data zakończenia zmiany.
- zadanie (VARCHAR2(255)): Opis zadania przypisanego do zmiany.

Powód istnienia:

Harmonogram pracy został oddzielony od danych statycznych o pracownikach.



- ## Relacje dla tabeli dok

  - **Relacja: dok → dok_nosnosc**
    Klucz obcy: dok_nosnosc.dokid → dok.dokid
    Znaczenie:
    Została utworzona, aby szczegóły dotyczące nośności doku (maksymalnanosnosc, liczbamiejsccumowniczych) mogły być przechowywane osobno.
    Relacja zapewnia możliwość zarządzania różnymi cechami nośności bez modyfikowania głównej tabeli dok.

  - **Relacja: dok → dok_stan**
    Klucz obcy: dok_stan.dokid → dok.dokid
    Znaczenie:
    Informacje o stanie doku (stan) zostały wydzielone, aby móc dynamicznie zmieniać jego status niezależnie od innych danych.

- ## Relacje dla tabeli harmonogramdokow

  - **Relacja: harmonogramdokow → harmonogramdokow_stan**
    Klucz obcy: harmonogramdokow_stan.harmonogramdokowid → harmonogramdokow.harmonogramdokowid
    Znaczenie:
    Relacja umożliwia przechowywanie dynamicznego statusu harmonogramu (stan) w osobnej tabeli, co poprawia elastyczność.

  - **Relacja: harmonogramdokow → harmonogramdokow_typ**
    Klucz obcy: harmonogramdokow_typ.harmonogramdokowid → harmonogramdokow.harmonogramdokowid
    Znaczenie:
    Pozwala na powiązanie harmonogramu z różnymi typami operacji (typoperacji), co umożliwia ich bardziej precyzyjne zarządzanie.

- ## Relacje dla tabeli kontener

  - **Relacja: kontener → kontener_lokalizacja**
    Klucz obcy: kontener_lokalizacja.kontenerid → kontener.kontenerid
    Znaczenie:
    Lokalizacje kontenera mogą być dynamicznie zmieniane i są przechowywane w osobnej tabeli.

  - **Relacja: kontener → kontener_status**
    Klucz obcy: kontener_status.kontenerid → kontener.kontenerid
    Znaczenie:
    Status kontenera (status) został wydzielony, aby umożliwić łatwą modyfikację bez ingerencji w podstawowe dane kontenera.

- ## Relacje dla tabeli przesylka

  - **Relacja: przesylka → przesylka_kontener**
    Klucz obcy: przesylka_kontener.przesylkaid → przesylka.przesylkaid
    Znaczenie:
    Umożliwia przypisanie wielu kontenerów do jednej przesyłki oraz precyzyjne zarządzanie ilościami (ilosc) i datami (datadodania).

  - **Relacja: przesylka → typprzesylki**
    Klucz obcy: przesylka.typprzesylkiid → typprzesylki.typprzesylkiid
    Znaczenie:
    Pozwala na określenie typu przesyłki (typprzesylki), co jest kluczowe dla zarządzania wymaganiami i specyfiką przesyłek.

  - **Relacja: przesylka → ubezpieczenie**
    Klucz obcy: przesylka.ubezpieczenieid → ubezpieczenie.ubezpieczenieid
    Znaczenie:
    Powiązanie przesyłki z jej ubezpieczeniem umożliwia przechowywanie i zarządzanie informacjami o ochronie transportowanej przesyłki.

- ## Relacje dla tabeli przeładunek

  - **Relacja: przeladunek → przeladunek_czas**
    Klucz obcy: przeladunek.przeladunek_czas_przeladunekid → przeladunek_czas.przeladunekid
    Znaczenie:
    Dane o czasie trwania przeładunku są zarządzane osobno, co pozwala na dokładniejszą analizę czasów operacji.

  - **Relacja: przeladunek → przeladunek_przesylka**
    Klucz obcy: przeladunek_przesylka.przeladunekid → przeladunek.przeladunekid
    Znaczenie:
    Relacja umożliwia przypisanie wielu przesyłek do jednego przeładunku, z uwzględnieniem czasu operacji i ilości przesyłek.

- ## Relacje dla tabeli ubezpieczenie

  - **Relacja: ubezpieczenie → typ_ubezpieczenia**
    Klucz obcy: typ_ubezpieczenia.ubezpieczenie_ubezpieczenieid → ubezpieczenie.ubezpieczenieid
    Znaczenie:
    Każde ubezpieczenie może być przypisane do jednego z wielu typów, co pozwala na różnicowanie zakresu i kosztów ubezpieczenia.

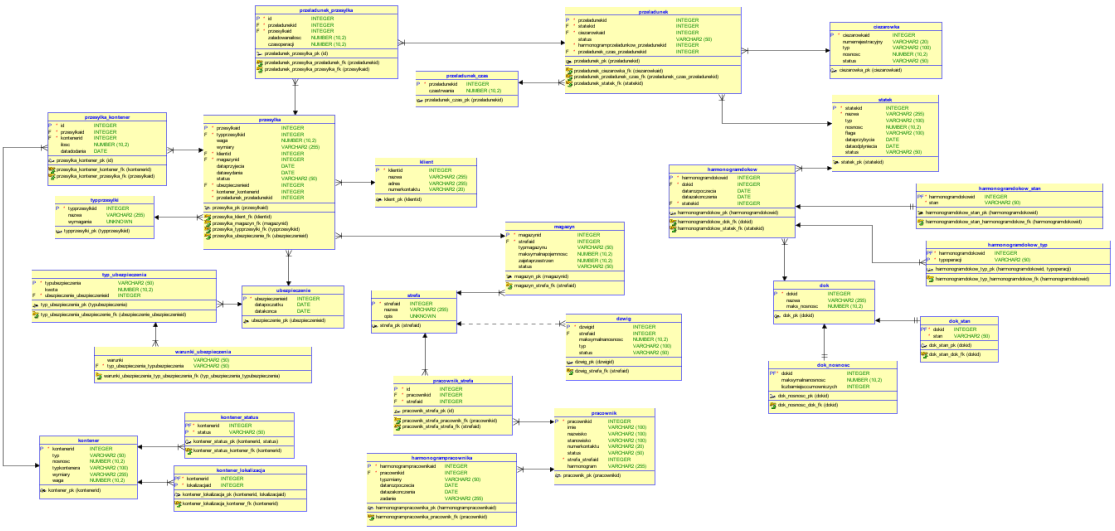
  - **Relacja: typ_ubezpieczenia → warunki_ubezpieczenia**
    Klucz obcy: warunki_ubezpieczenia.typ_ubezpieczenia_typubezpieczenia → typ_ubezpieczenia.typubezpieczenia
    Znaczenie:
    Umożliwia przypisanie warunków do konkretnych typów ubezpieczenia.

- ## Relacje dla tabeli pracownik

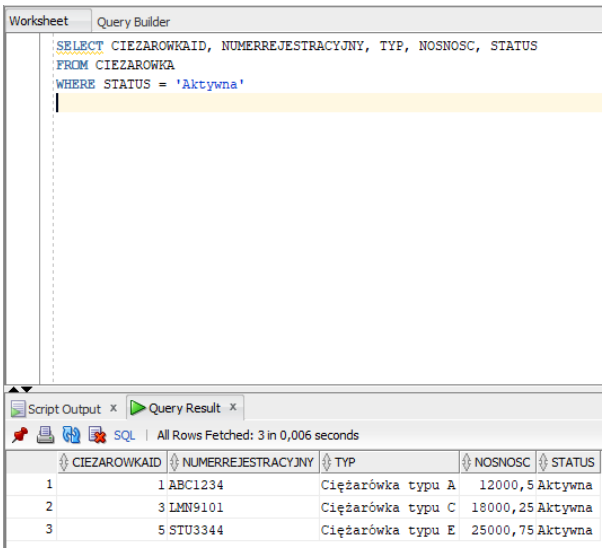
  - **Relacja: pracownik → pracownik_strefa**
    Klucz obcy: pracownik_strefa.pracownikid → pracownik.pracownikid
    Znaczenie:
    Relacja pozwala przypisać pracowników do wielu stref, umożliwiając elastyczne zarządzanie przypisaniami.

  - **Relacja: pracownik → harmonogrampracownika**
    Klucz obcy: harmonogrampracownika.pracownikid → pracownik.pracownikid
    Znaczenie:
    Przechowuje harmonogramy pracy pracowników w osobnej tabeli, co pozwala na bardziej szczegółowe planowanie.

2.4. Diagram relacyjny po normalizacji

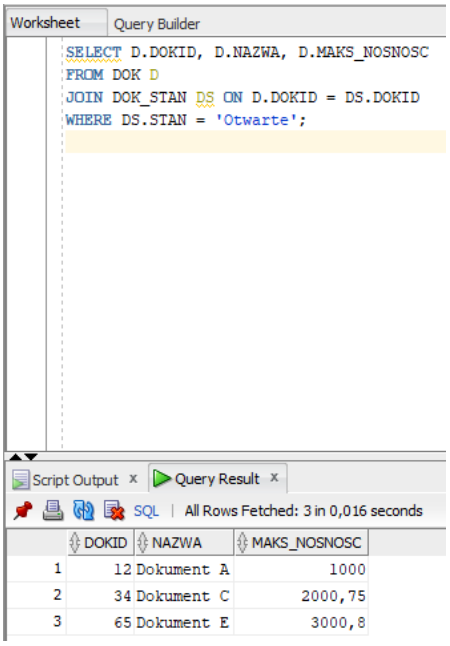


2.5. Operacje na danych



$$\pi\ CIEZAROWKA\ ID, NUMER_REJESTRACYJNY, TYP, NOSNOSC, STATUS\ (\sigma\ STATUS\ =\ 'Aktywny'\ (CIEZAROWKA))$$

Wybiera wszystkie aktywne ciężarówki, zwracając ich ID, numer rejestracyjny, typ, nośność oraz status.



$$\pi\ DOKID, NAZWA, MAKS_NOSNOSC\ (\sigma\ STAN\ =\ 'Otwarte'\ (DOK \bowtie DOK_STAN))$$

Wybiera dokumenty o statusie „Otwarte” i zwraca ich ID, nazwę oraz maksymalną nośność.

```
SELECT H.DOKID, D.NAZWA, H.DATAZAKONCZENIA
FROM HARMONOGRAMDOKOW H
JOIN DOK D ON H.DOKID = D.DOKID
WHERE H.DATAZAKONCZENIA > TO_DATE('2025-01-25', 'YYYY-MM-DD');
```

Script Output | Query Result

SQL | All Rows Fetched: 4 in 0,016 seconds

	DOKID	NAZWA	DATAZAKONCZENIA
1	2	Dok 2	25/01/26
2	3	Dok 3	25/01/27
3	4	Dok 4	25/01/28
4	5	Dok 5	25/01/29

$\pi\ DOKID, NAZWA, DATA_ZAKONCZENIA\ (\sigma\ DATA_ZAKONCZENI\ >\ '2025-01-25'\ (HARMONOGRAM_DOKOW \bowtie DOK))$

Wybiera dokumenty z harmonogramu, których data zakończenia jest po 25 stycznia 2025 roku, zwracając ich ID, nazwę i datę zakończenia.

```
SELECT DZWIGID, STREFAID, MAKSYMALNANOSNOSC, TYP, STATUS
FROM DZWIG
WHERE STREFAID > 103;
```

Script Output | Query Result

SQL | All Rows Fetched: 2 in 0,014 seconds

	DZWIGID	STREFAID	MAKSYMALNANOSNOSC	TYP	STATUS
1	4	104	8000	Typ D	W naprawie
2	5	105	10000	Typ E	Aktywny

$\pi\ DZWIGID, STREFAID, MAKSYMALNA_NOSNOSC, TYP, STATUS\ (\sigma\ STREFAID\ >\ 103\ (DZWIG))$

Wybiera dźwigi znajdujące się w strefach o ID większym niż 103, zwracając ich ID, ID strefy, maksymalną nośność, typ i status.

```
SELECT AVG(WAGA) AS SREDNIA_WAGA
FROM PRZESYLKA;
```

Script Output | Query Result

All Rows Fetched: 1 in 0,001 s

	SREDNIA_WAGA
1	244,625

$$\gamma\ SREDNIA_WAGA := AVG(WAGA)\ (PRZESYLKA)$$

Oblicza średnią wagę przesyłek.

```
SELECT MAGAZYNID, COUNT(*) AS LICZBA_PRZESYLEK
FROM PRZESYLKA
GROUP BY MAGAZYNID;
```

Script Output | Query Result

All Rows Fetched: 5 in 0,003 seconds

	MAGAZYNID	LICZBA_PRZESYLEK
1	1	2
2	2	1
3	3	2
4	4	2
5	5	1

$$\gamma\ MAGAZYNID, COUNT(*)\ (PRZESYLKA)$$

Zlicza liczbę przesyłek w każdym magazynie, grupując wyniki według ID magazynu.

```
SELECT MAX(MAKS_NOSNOSC) AS MAKSYMALNA_NOSNOSC
FROM DOK;
```

Script Output | Query Result

SQL | All Rows Fetched: 1 in 0,004 seconds

	MAKSYMALNA_NOSNOSC
1	25000

$$\gamma\ MAKSYMALNA_NOSNOSC := MAX(MAKS_NOSNOSC)\ (DOK)$$

Oblicza maksymalną nośność w tabeli dok.

```
SELECT TYP, SUM(NOSNOSC) AS SUMA_NOSNOSCI
FROM CIEZAROWKA
GROUP BY TYP;
```

Script Output | Query Result

SQL | All Rows Fetched: 5 in 0,004 seconds

	TYP	SUMA_NOSNOSCI
1	Ciężarówka typu A	12000,5
2	Ciężarówka typu B	15000
3	Ciężarówka typu C	18000,25
4	Ciężarówka typu D	20000
5	Ciężarówka typu E	25000,75

$$\gamma\ TYP, SUMA_NOSNOSCI := SUM(NOSNOSC)\ (CIEZAROWKA)$$

Grupuje ciężarówki według typu i oblicza sumę nośności dla każdego typu.

```sql
SELECT PR.PRZESYLKAID, PR.WAGA, PR.STATUS, K.NAZWA AS KLIENT, M.TYPMAGAZYNU AS MAGAZYN
FROM PRZESYLKA PR
JOIN KLIENT K ON PR.KLIENTID = K.KLIENTID
JOIN MAGAZYN M ON PR.MAGAZYNID = M.MAGAZYNID;
```

Script Output × Query Result ×

SQL | All Rows Fetched: 8 in 0,019 seconds

	PRZESYLKAID	WAGA	STATUS	KLIENT	MAGAZYN
1	1	150,5	W drodze	Electronics_1	Magazyn A
2	2	200	Zakończona	Food Products_2	Magazyn B
3	3	180,75	Wysłana	Goods_3	Magazyn C
4	4	225	W trakcie	Chemicals_4	Magazyn D
5	5	300	Oczekuje	Chemicals_5	Magazyn E
6	6	500,25	Wysłana	Electronics_1	Magazyn A
7	7	180	Zakończona	Food Products_2	Magazyn C
8	8	220,5	W drodze	Goods_3	Magazyn D

$$\pi\ PRZESYLKAID, WAGA, STATUS, NAZWA, TYP_MAGAZYNU\ ((PRZESYLKA \bowtie KLIENT) \bowtie MAGAZYN)$$

Łączy przesyłki z klientami i magazynami, zwracając ID przesyłki, wagę, status, nazwę klienta i typ magazynu.

```sql
SELECT P.PRZESYLKAID, P.WAGA, P.STATUS, K.TYP AS KONTENER, L.LOKALIZACJAID
FROM PRZESYLKA P
JOIN KONTENER K ON P.KONTENER_KONTENERID = K.KONTENERID
JOIN KONTENER_LOKALIZACJA L ON K.KONTENERID = L.KONTENERID;
```

Script Output × Query Result ×

SQL | All Rows Fetched: 8 in 0,02 seconds

	PRZESYLKAID	WAGA	STATUS	KONTENER	LOKALIZACJAID
1	1	150,5	W drodze	Typ A	101
2	2	200	Zakończona	Typ B	102
3	3	180,75	Wysłana	Typ C	103
4	4	225	W trakcie	Typ D	104
5	5	300	Oczekuje	Typ E	105
6	6	500,25	Wysłana	Typ A	101
7	7	180	Zakończona	Typ C	103
8	8	220,5	W drodze	Typ D	104

$$\pi\ PRZESYLKAID, WAGA, STATUS, TYP, LOKALIZACJAID\ ((PRZESYLKA \bowtie KONTENER) \bowtie KONTENER_LOKALIZACJA)$$

Łączy przesyłki z kontenerami i ich lokalizacjami, zwracając ID przesyłki, wagę, status, typ kontenera i ID lokalizacji.

```sql
SELECT M.MAGAZYNID, M.TYPMAGAZYNU, P.PRZESYLKAID, P.WAGA
FROM MAGAZYN M
JOIN PRZESYLKA P ON M.MAGAZYNID = P.MAGAZYNID
WHERE P.WAGA > 200;
```

Script Output | Query Result

SQL | All Rows Fetched: 4 in 0,005 seconds

	MAGAZYNID	TYPMAGAZYNU	PRZESYLKAID	WAGA
1	4	Magazyn D	4	225
2	5	Magazyn E	5	300
3	1	Magazyn A	6	500,25
4	4	Magazyn D	8	220,5

$$\pi\ MAGAZYNID, TYP_MAGAZYNU, PRZESYLKAID, WAGA\ (\sigma\ WAGA\ >\ 200\ (MAGAZYN \bowtie PRZESYLKA))$$

Wybiera przesyłki z magazynów, których waga jest większa niż 200, zwracając ID magazynu, typ magazynu, ID przesyłki i jej wagę.

```sql
SELECT K.KONTENERID, K.TYP, M.MAGAZYNID, M.TYPMAGAZYNU, M.ZAJETAPRZESTRZEN
FROM KONTENER K
JOIN MAGAZYN M ON K.KONTENERID = M.MAGAZYNID
WHERE M.ZAJETAPRZESTRZEN > 5000;
```

Script Output | Query Result

SQL | All Rows Fetched: 2 in 0,006 seconds

	KONTENERID	TYP	MAGAZYNID	TYPMAGAZYNU	ZAJETAPRZESTRZEN
1	4	Typ D	4	Magazyn D	5000,5
2	5	Typ E	5	Magazyn E	7000,75

$$\pi\ KONTENERID, TYP, MAGAZYNID, TYP_MAGAZYNU, ZAJETA_PRZESTRZEN\ (\sigma\ ZAJETA_PRZESTRZEN\ >\ 5000\ (KONTENER \bowtie MAGAZYN))$$

Wybiera kontenery w magazynach, gdzie zajęta przestrzeń przekracza 5000, zwracając ID kontenera, typ kontenera, ID magazynu, typ magazynu i zajętą przestrzeń.

3. Projekt implementacyjny

3.1. Rozwój aplikacji

- Dodawanie funkcji analitycznych oraz raportów dotyczących przesyłek, magazynów i efektywności operacyjnej.
- Rozwój systemu zarządzania alokacją zasobów, w tym doków, magazynów oraz kontenerów.
- Zwiększenie optymalizacji bazy danych poprzez wprowadzenie indeksów, archiwizacji i denormalizacji.
- Integracja z systemami zewnętrznymi, np. transportowymi i celno-logistycznymi, w celu usprawnienia przepływu informacji.
- Zabezpieczenie danych poprzez zaawansowane mechanizmy kontroli dostępu, szyfrowania oraz backupu.
- Dodanie możliwości śledzenia przesyłek w czasie rzeczywistym oraz zarządzania ich dokumentacją.

3.2. Kod bazy danych

```
CREATE TABLE ciezarowka (
   ciezarowkaid      INTEGER NOT NULL,
   numerrejestracyjny VARCHAR2(20),
   typ               VARCHAR2(100),
   nosnosc           NUMBER(10, 2),
   status            VARCHAR2(50)
)
LOGGING;

ALTER TABLE ciezarowka ADD CONSTRAINT ciezarowka_pk PRIMARY KEY (
ciezarowkaid );

CREATE TABLE dok (
   dokid       INTEGER NOT NULL,
   nazwa       VARCHAR2(255),
   maks_nosnosc NUMBER(10, 2)
)
LOGGING;

ALTER TABLE dok ADD CONSTRAINT dok_pk PRIMARY KEY ( dokid );

CREATE TABLE dok_nosnosc (
   dokid                  INTEGER NOT NULL,
   maksymalnanosnosc      NUMBER(10, 2),
   liczbamiejsccumowniczych INTEGER
)
LOGGING;

ALTER TABLE dok_nosnosc ADD CONSTRAINT dok_nosnosc_pk PRIMARY KEY ( dokid );

CREATE TABLE dok_stan (
```

```
    dokid INTEGER NOT NULL,
    stan  VARCHAR2(50) NOT NULL
)
LOGGING;

ALTER TABLE dok_stan ADD CONSTRAINT dok_stan_pk PRIMARY KEY ( dokid );

CREATE TABLE dzwig (
    dzwigid          INTEGER NOT NULL,
    strefaid         INTEGER,
    maksymalnanosnosc NUMBER(10, 2),
    typ              VARCHAR2(100),
    status           VARCHAR2(50)
)
LOGGING;

ALTER TABLE dzwig ADD CONSTRAINT dzwig_pk PRIMARY KEY ( dzwigid );

CREATE TABLE harmonogramdokow (
    harmonogramdokowid INTEGER NOT NULL,
    dokid              INTEGER NOT NULL,
    datarozpoczecia    DATE,
    datazakonczenia    DATE,
    statekid           INTEGER NOT NULL
)
LOGGING;

ALTER TABLE harmonogramdokow ADD CONSTRAINT harmonogramdokow_pk PRIMARY
KEY ( harmonogramdokowid );

CREATE TABLE harmonogramdokow_stan (
    harmonogramdokowid INTEGER NOT NULL,
    stan               VARCHAR2(50) NOT NULL
)
LOGGING;

ALTER TABLE harmonogramdokow_stan ADD CONSTRAINT harmonogramdokow_stan_pk
PRIMARY KEY ( harmonogramdokowid );

CREATE TABLE harmonogramdokow_typ (
    harmonogramdokowid INTEGER NOT NULL,
    typoperacji        VARCHAR2(50) NOT NULL
)
LOGGING;

ALTER TABLE harmonogramdokow_typ ADD CONSTRAINT harmonogramdokow_typ_pk
PRIMARY KEY ( harmonogramdokowid,
                                                    typoperacji );

CREATE TABLE harmonogrampracownika (
    harmonogrampracownikaid INTEGER NOT NULL,
    pracownikid             INTEGER NOT NULL,
    typzmiany               VARCHAR2(50),
    datarozpoczecia         DATE,
    datazakonczenia         DATE,
    zadanie                 VARCHAR2(255)
)
```

```
LOGGING;

ALTER TABLE harmonogrampracownika ADD CONSTRAINT harmonogrampracownika_pk
PRIMARY KEY ( harmonogrampracownikaid );

CREATE TABLE klient (
   klientid      INTEGER NOT NULL,
   nazwa         VARCHAR2(255),
   adres         VARCHAR2(255),
   numerkontaktu VARCHAR2(20)
)
LOGGING;

ALTER TABLE klient ADD CONSTRAINT klient_pk PRIMARY KEY ( klientid );

CREATE TABLE kontener (
   kontenerid   INTEGER NOT NULL,
   typ          VARCHAR2(50),
   nosnosc      NUMBER(10, 2),
   typkontenera VARCHAR2(100),
   wymiary      VARCHAR2(255),
   waga         NUMBER(10, 2)
)
LOGGING;

ALTER TABLE kontener ADD CONSTRAINT kontener_pk PRIMARY KEY ( kontenerid );

CREATE TABLE kontener_lokalizacja (
   kontenerid    INTEGER NOT NULL,
   lokalizacjaid INTEGER NOT NULL
)
LOGGING;

ALTER TABLE kontener_lokalizacja ADD CONSTRAINT kontener_lokalizacja_pk PRIMARY
KEY ( kontenerid,
                                                    lokalizacjaid );

CREATE TABLE kontener_status (
   kontenerid INTEGER NOT NULL,
   status     VARCHAR2(50) NOT NULL
)
LOGGING;

ALTER TABLE kontener_status ADD CONSTRAINT kontener_status_pk PRIMARY KEY (
kontenerid,
                                                 status );

CREATE TABLE magazyn (
   magazynid           INTEGER NOT NULL,
   strefaid            INTEGER NOT NULL,
   typmagazynu         VARCHAR2(50),
   maksymalnapojemnosc NUMBER(10, 2),
   zajetaprzestrzen    NUMBER(10, 2),
   status              VARCHAR2(50)
)
LOGGING;
```

```
ALTER TABLE magazyn ADD CONSTRAINT magazyn_pk PRIMARY KEY ( magazynid );

CREATE TABLE pracownik (
   pracownikid     INTEGER NOT NULL,
   imie            VARCHAR2(100),
   nazwisko        VARCHAR2(100),
   stanowisko      VARCHAR2(100),
   numerkontaktu   VARCHAR2(20),
   status          VARCHAR2(50),
   strefa_strefaid INTEGER NOT NULL,
   harmonogram     VARCHAR2(255)
)
LOGGING;

ALTER TABLE pracownik ADD CONSTRAINT pracownik_pk PRIMARY KEY ( pracownikid
);

CREATE TABLE pracownik_strefa (
   id          INTEGER NOT NULL,
   pracownikid INTEGER NOT NULL,
   strefaid    INTEGER NOT NULL
)
LOGGING;

ALTER TABLE pracownik_strefa ADD CONSTRAINT pracownik_strefa_pk PRIMARY KEY (
id );

CREATE TABLE przeladunek (
   przeladunekid                    INTEGER NOT NULL,
   statekid                         INTEGER NOT NULL,
   ciezarowkaid                     INTEGER NOT NULL,
   status                           VARCHAR2(50),
   harmonogramprzeladunkow_przeladunekid INTEGER NOT NULL,
   przeladunek_czas_przeladunekid       INTEGER NOT NULL
)
LOGGING;

ALTER TABLE przeladunek ADD CONSTRAINT przeladunek_pk PRIMARY KEY (
przeladunekid );

CREATE TABLE przeladunek_czas (
   przeladunekid INTEGER NOT NULL,
   czastrwania   NUMBER(10, 2)
)
LOGGING;

ALTER TABLE przeladunek_czas ADD CONSTRAINT przeladunek_czas_pk PRIMARY KEY
( przeladunekid );

CREATE TABLE przeladunek_przesylka (
   id              INTEGER NOT NULL,
   przeladunekid   INTEGER NOT NULL,
   przesylkaid     INTEGER NOT NULL,
   zaladowanailosc NUMBER(10, 2),
   czasoperacji    NUMBER(10, 2)
)
LOGGING;
```

```
ALTER TABLE przeladunek_przesylka ADD CONSTRAINT przeladunek_przesylka_pk
PRIMARY KEY ( id );

CREATE TABLE przesylka (
   przesylkaid              INTEGER NOT NULL,
   typprzesylkiid           INTEGER NOT NULL,
   waga                     NUMBER(10, 2),
   wymiary                  VARCHAR2(255),
   klientid                 INTEGER NOT NULL,
   magazynid                INTEGER NOT NULL,
   dataprzyjecia            DATE,
   datawydania              DATE,
   status                   VARCHAR2(50),
   ubezpieczenieid          INTEGER NOT NULL,
   kontener_kontenerid      INTEGER NOT NULL,
   przeladunek_przeladunekid INTEGER NOT NULL
)
LOGGING;

ALTER TABLE przesylka ADD CONSTRAINT przesylka_pk PRIMARY KEY ( przesylkaid );

CREATE TABLE przesylka_kontener (
   id          INTEGER NOT NULL,
   przesylkaid INTEGER NOT NULL,
   kontenerid  INTEGER NOT NULL,
   ilosc       NUMBER(10, 2),
   datadodania DATE
)
LOGGING;

ALTER TABLE przesylka_kontener ADD CONSTRAINT przesylka_kontener_pk PRIMARY
KEY ( id );

CREATE TABLE statek (
   statekid        INTEGER NOT NULL,
   nazwa           VARCHAR2(255) NOT NULL,
   typ             VARCHAR2(100),
   nosnosc         NUMBER(10, 2),
   flaga           VARCHAR2(100),
   dataprzybycia   DATE,
   dataodplyniecia DATE,
   status          VARCHAR2(50)
)
LOGGING;

ALTER TABLE statek ADD CONSTRAINT statek_pk PRIMARY KEY ( statekid );

CREATE TABLE strefa (
   strefaid INTEGER NOT NULL,
   nazwa    VARCHAR2(255),
   opis     unknown
--  ERROR: Datatype UNKNOWN is not allowed

)
LOGGING;
```

```
ALTER TABLE strefa ADD CONSTRAINT strefa_pk PRIMARY KEY ( strefaid );

CREATE TABLE typ_ubezpieczenia (
   typubezpieczenia            VARCHAR2(50) NOT NULL,
   kwota                       NUMBER(10, 2),
   ubezpieczenie_ubezpieczenieid INTEGER NOT NULL
)
LOGGING;

ALTER TABLE typ_ubezpieczenia ADD CONSTRAINT typ_ubezpieczenia_pk PRIMARY
KEY ( typubezpieczenia );

CREATE TABLE typprzesylki (
   typprzesylkiid INTEGER NOT NULL,
   nazwa          VARCHAR2(255),
   wymagania      unknown
--  ERROR: Datatype UNKNOWN is not allowed

)
LOGGING;

ALTER TABLE typprzesylki ADD CONSTRAINT typprzesylki_pk PRIMARY KEY (
typprzesylkiid );

CREATE TABLE ubezpieczenie (
   ubezpieczenieid INTEGER NOT NULL,
   datapoczatku    DATE,
   datakonca       DATE
)
LOGGING;

ALTER TABLE ubezpieczenie ADD CONSTRAINT ubezpieczenie_pk PRIMARY KEY (
ubezpieczenieid );

CREATE TABLE warunki_ubezpieczenia (
   warunki                        VARCHAR2(50),
   typ_ubezpieczenia_typubezpieczenia VARCHAR2(50) NOT NULL
)
LOGGING;

ALTER TABLE dok_nosnosc
   ADD CONSTRAINT dok_nosnosc_dok_fk FOREIGN KEY ( dokid )
      REFERENCES dok ( dokid )
   NOT DEFERRABLE;

ALTER TABLE dok_stan
   ADD CONSTRAINT dok_stan_dok_fk FOREIGN KEY ( dokid )
      REFERENCES dok ( dokid )
   NOT DEFERRABLE;

ALTER TABLE dzwig
   ADD CONSTRAINT dzwig_strefa_fk FOREIGN KEY ( strefaid )
      REFERENCES strefa ( strefaid )
   NOT DEFERRABLE;

ALTER TABLE harmonogramdokow
   ADD CONSTRAINT harmonogramdokow_dok_fk FOREIGN KEY ( dokid )
```

```
      REFERENCES dok ( dokid )
   NOT DEFERRABLE;

ALTER TABLE harmonogramdokow_stan
   ADD CONSTRAINT harmonogramdokow_stan_harmonogramdokow_fk FOREIGN KEY (
harmonogramdokowid )
      REFERENCES harmonogramdokow ( harmonogramdokowid )
   NOT DEFERRABLE;

ALTER TABLE harmonogramdokow
   ADD CONSTRAINT harmonogramdokow_statek_fk FOREIGN KEY ( statekid )
      REFERENCES statek ( statekid )
   NOT DEFERRABLE;

ALTER TABLE harmonogramdokow_typ
   ADD CONSTRAINT harmonogramdokow_typ_harmonogramdokow_fk FOREIGN KEY (
harmonogramdokowid )
      REFERENCES harmonogramdokow ( harmonogramdokowid )
   NOT DEFERRABLE;

ALTER TABLE harmonogrampracownika
   ADD CONSTRAINT harmonogrampracownika_pracownik_fk FOREIGN KEY ( pracownikid
)
      REFERENCES pracownik ( pracownikid )
   NOT DEFERRABLE;

ALTER TABLE kontener_lokalizacja
   ADD CONSTRAINT kontener_lokalizacja_kontener_fk FOREIGN KEY ( kontenerid )
      REFERENCES kontener ( kontenerid )
   NOT DEFERRABLE;

ALTER TABLE kontener_status
   ADD CONSTRAINT kontener_status_kontener_fk FOREIGN KEY ( kontenerid )
      REFERENCES kontener ( kontenerid )
   NOT DEFERRABLE;

ALTER TABLE magazyn
   ADD CONSTRAINT magazyn_strefa_fk FOREIGN KEY ( strefaid )
      REFERENCES strefa ( strefaid )
   NOT DEFERRABLE;

ALTER TABLE pracownik_strefa
   ADD CONSTRAINT pracownik_strefa_pracownik_fk FOREIGN KEY ( pracownikid )
      REFERENCES pracownik ( pracownikid )
   NOT DEFERRABLE;

ALTER TABLE pracownik_strefa
   ADD CONSTRAINT pracownik_strefa_strefa_fk FOREIGN KEY ( strefaid )
      REFERENCES strefa ( strefaid )
   NOT DEFERRABLE;

ALTER TABLE przeladunek
   ADD CONSTRAINT przeladunek_ciezarowka_fk FOREIGN KEY ( ciezarowkaid )
      REFERENCES ciezarowka ( ciezarowkaid )
   NOT DEFERRABLE;

ALTER TABLE przeladunek
```

```
   ADD CONSTRAINT przeladunek_przeladunek_czas_fk FOREIGN KEY (
przeladunek_czas_przeladunekid )
      REFERENCES przeladunek_czas ( przeladunekid )
   NOT DEFERRABLE;

ALTER TABLE przeladunek_przesylka
   ADD CONSTRAINT przeladunek_przesylka_przeladunek_fk FOREIGN KEY ( przeladunekid
)
      REFERENCES przeladunek ( przeladunekid )
   NOT DEFERRABLE;

ALTER TABLE przeladunek_przesylka
   ADD CONSTRAINT przeladunek_przesylka_przesylka_fk FOREIGN KEY ( przesylkaid )
      REFERENCES przesylka ( przesylkaid )
   NOT DEFERRABLE;

ALTER TABLE przeladunek
   ADD CONSTRAINT przeladunek_statek_fk FOREIGN KEY ( statekid )
      REFERENCES statek ( statekid )
   NOT DEFERRABLE;

ALTER TABLE przesylka
   ADD CONSTRAINT przesylka_klient_fk FOREIGN KEY ( klientid )
      REFERENCES klient ( klientid )
   NOT DEFERRABLE;

ALTER TABLE przesylka_kontener
   ADD CONSTRAINT przesylka_kontener_kontener_fk FOREIGN KEY ( kontenerid )
      REFERENCES kontener ( kontenerid )
   NOT DEFERRABLE;

ALTER TABLE przesylka_kontener
   ADD CONSTRAINT przesylka_kontener_przesylka_fk FOREIGN KEY ( przesylkaid )
      REFERENCES przesylka ( przesylkaid )
   NOT DEFERRABLE;

ALTER TABLE przesylka
   ADD CONSTRAINT przesylka_magazyn_fk FOREIGN KEY ( magazynid )
      REFERENCES magazyn ( magazynid )
   NOT DEFERRABLE;

ALTER TABLE przesylka
   ADD CONSTRAINT przesylka_typprzesylki_fk FOREIGN KEY ( typprzesylkiid )
      REFERENCES typprzesylki ( typprzesylkiid )
   NOT DEFERRABLE;

ALTER TABLE przesylka
   ADD CONSTRAINT przesylka_ubezpieczenie_fk FOREIGN KEY ( ubezpieczenieid )
      REFERENCES ubezpieczenie ( ubezpieczenieid )
   NOT DEFERRABLE;

ALTER TABLE typ_ubezpieczenia
   ADD CONSTRAINT typ_ubezpieczenia_ubezpieczenie_fk FOREIGN KEY (
ubezpieczenie_ubezpieczenieid )
      REFERENCES ubezpieczenie ( ubezpieczenieid )
   NOT DEFERRABLE;
```

```
ALTER TABLE warunki_ubezpieczenia
   ADD CONSTRAINT warunki_ubezpieczenia_typ_ubezpieczenia_fk FOREIGN KEY (
typ_ubezpieczenia_typubezpieczenia )
      REFERENCES typ_ubezpieczenia ( typubezpieczenia )
   NOT DEFERRABLE;
```

3.3. Wprowadzanie danych do bazy.

```
inserty:
ciezarowka
INSERT INTO ciezarowka (CIEZAROWKAID, NUMERREJESTRACYJNY, TYP, NOSNOSC,
STATUS)
VALUES (1, 'ABC1234', 'Ciężarówka typu A', 12000.50, 'Aktywna');

INSERT INTO ciezarowka (CIEZAROWKAID, NUMERREJESTRACYJNY, TYP, NOSNOSC,
STATUS)
VALUES (2, 'XYZ5678', 'Ciężarówka typu B', 15000.00, 'W naprawie');

INSERT INTO ciezarowka (CIEZAROWKAID, NUMERREJESTRACYJNY, TYP, NOSNOSC,
STATUS)
VALUES (3, 'LMN9101', 'Ciężarówka typu C', 18000.25, 'Aktywna');

INSERT INTO ciezarowka (CIEZAROWKAID, NUMERREJESTRACYJNY, TYP, NOSNOSC,
STATUS)
VALUES (4, 'PQR1122', 'Ciężarówka typu D', 20000.00, 'W naprawie');

INSERT INTO ciezarowka (CIEZAROWKAID, NUMERREJESTRACYJNY, TYP, NOSNOSC,
STATUS)
VALUES (5, 'STU3344', 'Ciężarówka typu E', 25000.75, 'Aktywna');
----------
dok
INSERT INTO dok (dokid, nazwa, maks_nosnosc)
VALUES (12, 'Dokument A', 1000.00);

INSERT INTO dok (dokid, nazwa, maks_nosnosc)
VALUES (23, 'Dokument B', 1500.50);

INSERT INTO dok (dokid, nazwa, maks_nosnosc)
VALUES (34, 'Dokument C', 2000.75);

INSERT INTO dok (dokid, nazwa, maks_nosnosc)
VALUES (45, 'Dokument D', 2500.25);

INSERT INTO dok (dokid, nazwa, maks_nosnosc)
VALUES (65, 'Dokument E', 3000.80);
-------
dok_nosnosc
INSERT INTO DOK_NOSNOSC (DOKID, MAKSYMALNANOSNOSC,
LICZBAMIEJSCCUMOWNICZYCH)
VALUES (12, 10000.00, 10);

INSERT INTO DOK_NOSNOSC (DOKID, MAKSYMALNANOSNOSC,
LICZBAMIEJSCCUMOWNICZYCH)
VALUES (23, 12000.50, 15);

INSERT INTO DOK_NOSNOSC (DOKID, MAKSYMALNANOSNOSC,
LICZBAMIEJSCCUMOWNICZYCH)
VALUES (34, 13000.75, 20);

INSERT INTO DOK_NOSNOSC (DOKID, MAKSYMALNANOSNOSC,
LICZBAMIEJSCCUMOWNICZYCH)
VALUES (45, 14000.25, 25);
```

```
INSERT INTO DOK_NOSNOSC (DOKID, MAKSYMALNANOSNOSC,
LICZBAMIEJSCCUMOWNICZYCH)
VALUES (65, 15000.80, 30);
-----
dok_stan
INSERT INTO DOK_STAN (DOKID, STAN)
VALUES (12, 'Otwarte');

INSERT INTO DOK_STAN (DOKID, STAN)
VALUES (23, 'Zamknięte');

INSERT INTO DOK_STAN (DOKID, STAN)
VALUES (34, 'Otwarte');

INSERT INTO DOK_STAN (DOKID, STAN)
VALUES (45, 'Zamknięte');

INSERT INTO DOK_STAN (DOKID, STAN)
VALUES (65, 'Otwarte');
-----
dzwig
INSERT INTO dzwig (dzwigid, strefaid, maksymalnanosnosc, typ, status)
VALUES (1, 101, 5000.00, 'Typ A', 'Aktywny');

INSERT INTO dzwig (dzwigid, strefaid, maksymalnanosnosc, typ, status)
VALUES (2, 102, 7000.00, 'Typ B', 'W naprawie');

INSERT INTO dzwig (dzwigid, strefaid, maksymalnanosnosc, typ, status)
VALUES (3, 103, 6000.00, 'Typ C', 'Aktywny');

INSERT INTO dzwig (dzwigid, strefaid, maksymalnanosnosc, typ, status)
VALUES (4, 104, 8000.00, 'Typ D', 'W naprawie');

INSERT INTO dzwig (dzwigid, strefaid, maksymalnanosnosc, typ, status)
VALUES (5, 105, 10000.00, 'Typ E', 'Aktywny');
------harmonogram dokow
INSERT INTO harmonogramdokow (harmonogramdokowid, dokid, datarozpoczecia,
datazakonczenia, statekid)
VALUES (12, 1, TO_DATE('2025-01-20', 'YYYY-MM-DD'), TO_DATE('2025-01-25', 'YYYY-
MM-DD'), 1);

INSERT INTO harmonogramdokow (harmonogramdokowid, dokid, datarozpoczecia,
datazakonczenia, statekid)
VALUES (23, 2, TO_DATE('2025-01-21', 'YYYY-MM-DD'), TO_DATE('2025-01-26', 'YYYY-
MM-DD'), 2);

INSERT INTO harmonogramdokow (harmonogramdokowid, dokid, datarozpoczecia,
datazakonczenia, statekid)
VALUES (34, 3, TO_DATE('2025-01-22', 'YYYY-MM-DD'), TO_DATE('2025-01-27', 'YYYY-
MM-DD'), 3);

INSERT INTO harmonogramdokow (harmonogramdokowid, dokid, datarozpoczecia,
datazakonczenia, statekid)
VALUES (45, 4, TO_DATE('2025-01-23', 'YYYY-MM-DD'), TO_DATE('2025-01-28', 'YYYY-
MM-DD'), 4);
```

```
INSERT INTO harmonogramdokow (harmonogramdokowid, dokid, datarozpoczecia,
datazakonczenia, statekid)
VALUES (65, 5, TO_DATE('2025-01-24', 'YYYY-MM-DD'), TO_DATE('2025-01-29', 'YYYY-
MM-DD'), 5);

------
statek
INSERT INTO statek (statekid, nazwa, typ, nosnosc, flaga, dataprzybycia, dataodplyniecia, status)
VALUES (1, 'Statek A', 'Typ A', 10000.00, 'Polska', TO_DATE('2025-01-10', 'YYYY-MM-DD'),
TO_DATE('2025-01-20', 'YYYY-MM-DD'), 'Aktywny');

INSERT INTO statek (statekid, nazwa, typ, nosnosc, flaga, dataprzybycia, dataodplyniecia, status)
VALUES (2, 'Statek B', 'Typ B', 15000.50, 'Niemcy', TO_DATE('2025-01-12', 'YYYY-MM-DD'),
TO_DATE('2025-01-22', 'YYYY-MM-DD'), 'W naprawie');

INSERT INTO statek (statekid, nazwa, typ, nosnosc, flaga, dataprzybycia, dataodplyniecia, status)
VALUES (3, 'Statek C', 'Typ C', 20000.75, 'Francja', TO_DATE('2025-01-14', 'YYYY-MM-DD'),
TO_DATE('2025-01-24', 'YYYY-MM-DD'), 'Aktywny');

INSERT INTO statek (statekid, nazwa, typ, nosnosc, flaga, dataprzybycia, dataodplyniecia, status)
VALUES (4, 'Statek D', 'Typ D', 25000.25, 'Hiszpania', TO_DATE('2025-01-16', 'YYYY-MM-
DD'), TO_DATE('2025-01-26', 'YYYY-MM-DD'), 'W naprawie');

INSERT INTO statek (statekid, nazwa, typ, nosnosc, flaga, dataprzybycia, dataodplyniecia, status)
VALUES (5, 'Statek E', 'Typ E', 30000.00, 'Wielka Brytania', TO_DATE('2025-01-18', 'YYYY-
MM-DD'), TO_DATE('2025-01-28', 'YYYY-MM-DD'), 'Aktywny');
-----
harmonogramdokow_stan

INSERT INTO harmonogramdokow_stan (harmonogramdokowid, stan)
VALUES (12, 'W trakcie');

INSERT INTO harmonogramdokow_stan (harmonogramdokowid, stan)
VALUES (23, 'Zakończono');

INSERT INTO harmonogramdokow_stan (harmonogramdokowid, stan)
VALUES (34, 'W trakcie');

INSERT INTO harmonogramdokow_stan (harmonogramdokowid, stan)
VALUES (45, 'Zakończono');

INSERT INTO harmonogramdokow_stan (harmonogramdokowid, stan)
VALUES (65, 'W trakcie');
-------
harmonoggramdokow_typ
INSERT INTO harmonogramdokow_typ (harmonogramdokowid, typoperacji)
VALUES (12, 'Załadunek');

INSERT INTO harmonogramdokow_typ (harmonogramdokowid, typoperacji)
VALUES (23, 'Rozładunek');

INSERT INTO harmonogramdokow_typ (harmonogramdokowid, typoperacji)
VALUES (34, 'Przegląd techniczny');

INSERT INTO harmonogramdokow_typ (harmonogramdokowid, typoperacji)
VALUES (45, 'Załadunek');
```

```
INSERT INTO harmonogramdokow_typ (harmonogramdokowid, typoperacji)
VALUES (65, 'Rozładunek');
-------
klient
INSERT INTO klient (klientid, nazwa, adres, numerkontaktu)
VALUES (1, 'Electronics_1', 'Warszawa_12', 'NUM-8308');

INSERT INTO klient (klientid, nazwa, adres, numerkontaktu)
VALUES (2, 'Food Products_2', 'Szczecin_17', 'NUM-9928');

INSERT INTO klient (klientid, nazwa, adres, numerkontaktu)
VALUES (3, 'Goods_3', 'Łódź_32', 'NUM-3714');

INSERT INTO klient (klientid, nazwa, adres, numerkontaktu)
VALUES (4, 'Chemicals_4', 'Łódź_30', 'NUM-1215');

INSERT INTO klient (klientid, nazwa, adres, numerkontaktu)
VALUES (5, 'Chemicals_5', 'Łódź_34', 'NUM-8401');
-----
kontener
INSERT INTO kontener (kontenerid, typ, nosnosc, typkontenera, wymiary, waga)
VALUES (1, 'Typ A', 1000.00, 'Kontener standardowy', '6x2x2', 2000.50);

INSERT INTO kontener (kontenerid, typ, nosnosc, typkontenera, wymiary, waga)
VALUES (2, 'Typ B', 1500.50, 'Kontener chłodniczy', '6x2x2.5', 2500.75);

INSERT INTO kontener (kontenerid, typ, nosnosc, typkontenera, wymiary, waga)
VALUES (3, 'Typ C', 2000.25, 'Kontener otwarty', '7x2.5x3', 3000.00);

INSERT INTO kontener (kontenerid, typ, nosnosc, typkontenera, wymiary, waga)
VALUES (4, 'Typ D', 2500.00, 'Kontener zamknięty', '8x3x3', 3500.25);

INSERT INTO kontener (kontenerid, typ, nosnosc, typkontenera, wymiary, waga)
VALUES (5, 'Typ E', 3000.75, 'Kontener specjalistyczny', '6x3x3', 4000.00);
-----  kontener_lokalizacja
INSERT INTO kontener_lokalizacja (kontenerid, lokalizacjaid)
VALUES (1, 101);

INSERT INTO kontener_lokalizacja (kontenerid, lokalizacjaid)
VALUES (2, 102);

INSERT INTO kontener_lokalizacja (kontenerid, lokalizacjaid)
VALUES (3, 103);

INSERT INTO kontener_lokalizacja (kontenerid, lokalizacjaid)
VALUES (4, 104);

INSERT INTO kontener_lokalizacja (kontenerid, lokalizacjaid)
VALUES (5, 105);
--------
kontener_status
INSERT INTO kontener_status (kontenerid, status)
VALUES (1, 'Dostarczony');

INSERT INTO kontener_status (kontenerid, status)
VALUES (2, 'W drodze');
```

```
INSERT INTO kontener_status (kontenerid, status)
VALUES (3, 'Oczekujący');

INSERT INTO kontener_status (kontenerid, status)
VALUES (4, 'Załadowany');

INSERT INTO kontener_status (kontenerid, status)
VALUES (5, 'W magazynie');
------
magazyn
INSERT INTO magazyn (magazynid, strefaid, typmagazynu, maksymalnapojemnosc,
zajetaprzestrzen, status)
VALUES (1, 1, 'Magazyn A', 5000.00, 1000.50, 'Aktywny');

INSERT INTO magazyn (magazynid, strefaid, typmagazynu, maksymalnapojemnosc,
zajetaprzestrzen, status)
VALUES (2, 2, 'Magazyn B', 10000.00, 2500.75, 'W naprawie');

INSERT INTO magazyn (magazynid, strefaid, typmagazynu, maksymalnapojemnosc,
zajetaprzestrzen, status)
VALUES (3, 1, 'Magazyn C', 8000.50, 3000.00, 'Aktywny');

INSERT INTO magazyn (magazynid, strefaid, typmagazynu, maksymalnapojemnosc,
zajetaprzestrzen, status)
VALUES (4, 3, 'Magazyn D', 12000.00, 5000.50, 'Zamknięty');

INSERT INTO magazyn (magazynid, strefaid, typmagazynu, maksymalnapojemnosc,
zajetaprzestrzen, status)
VALUES (5, 2, 'Magazyn E', 15000.00, 7000.75, 'W naprawie');
-------- pracownik
INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status,
strefa_strefaid)
VALUES (1, 'Ewa', 'Szymański', 1, 'NUM-5688', 'Inactive', 1);

INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status,
strefa_strefaid)
VALUES (2, 'Jan', 'Kamiński', 2, 'NUM-2424', 'Inactive', 2);

INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status,
strefa_strefaid)
VALUES (3, 'Marek', 'Szymański', 3, 'NUM-1760', 'Completed', 3);

INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status,
strefa_strefaid)
VALUES (4, 'Marek', 'Wiśniewski', 4, 'NUM-4578', 'Completed', 4);

INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status,
strefa_strefaid)
VALUES (5, 'Jan', 'Lewandowski', 5, 'NUM-2070', 'Pending', 5);

INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status,
strefa_strefaid)
VALUES (6, 'Ewa', 'Zieliński', 6, 'NUM-6680', 'Active', 6);

INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status,
strefa_strefaid)
VALUES (7, 'Anna', 'Wójcik', 7, 'NUM-5948', 'Inactive', 7);
```

```
INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status,
strefa_strefaid)
VALUES (8, 'Anna', 'Wójcik', 8, 'NUM-1206', 'Failed', 8);
-----harmonoggram pracownika
INSERT INTO harmonogrampracownika (harmonogrampracownikaid, pracownikid, typzmiany,
datarozpoczecia, datazakonczenia, zadanie)
VALUES (1, 1, 'Ranna', TO_DATE('2025-01-15', 'YYYY-MM-DD'), TO_DATE('2025-01-15',
'YYYY-MM-DD'), 'Pakowanie');

INSERT INTO harmonogrampracownika (harmonogrampracownikaid, pracownikid, typzmiany,
datarozpoczecia, datazakonczenia, zadanie)
VALUES (2, 2, 'Popołudniowa', TO_DATE('2025-01-15', 'YYYY-MM-DD'), TO_DATE('2025-
01-15', 'YYYY-MM-DD'), 'Załadunek');

INSERT INTO harmonogrampracownika (harmonogrampracownikaid, pracownikid, typzmiany,
datarozpoczecia, datazakonczenia, zadanie)
VALUES (3, 3, 'Nocna', TO_DATE('2025-01-15', 'YYYY-MM-DD'), TO_DATE('2025-01-15',
'YYYY-MM-DD'), 'Załadunek');

INSERT INTO harmonogrampracownika (harmonogrampracownikaid, pracownikid, typzmiany,
datarozpoczecia, datazakonczenia, zadanie)
VALUES (4, 4, 'Ranna', TO_DATE('2025-01-15', 'YYYY-MM-DD'), TO_DATE('2025-01-15',
'YYYY-MM-DD'), 'Pakowanie');

INSERT INTO harmonogrampracownika (harmonogrampracownikaid, pracownikid, typzmiany,
datarozpoczecia, datazakonczenia, zadanie)
VALUES (5, 5, 'Popołudniowa', TO_DATE('2025-01-15', 'YYYY-MM-DD'), TO_DATE('2025-
01-15', 'YYYY-MM-DD'), 'Transport');
--------pracownik strefa
INSERT INTO PRACOWNIK (PRACOWNIKID, IMIE, NAZWISKO, STANOWISKO,
NUMERKONTAKTU, STATUS, STREFA_STREFAID)
VALUES (1, 'Ewa', 'Szymańska', 'Kierownik', 'NUM-1234', 'Aktywny', 1);

INSERT INTO PRACOWNIK (PRACOWNIKID, IMIE, NAZWISKO, STANOWISKO,
NUMERKONTAKTU, STATUS, STREFA_STREFAID)
VALUES (2, 'Jan', 'Kamiński', 'Magazynier', 'NUM-5678', 'W trakcie', 2);

INSERT INTO PRACOWNIK (PRACOWNIKID, IMIE, NAZWISKO, STANOWISKO,
NUMERKONTAKTU, STATUS, STREFA_STREFAID)
VALUES (3, 'Marek', 'Wiśniewski', 'Operator', 'NUM-3456', 'Zakończony', 3);

INSERT INTO PRACOWNIK (PRACOWNIKID, IMIE, NAZWISKO, STANOWISKO,
NUMERKONTAKTU, STATUS, STREFA_STREFAID)
VALUES (4, 'Anna', 'Lewandowska', 'Asystentka', 'NUM-7890', 'Aktywny', 4);

INSERT INTO PRACOWNIK (PRACOWNIKID, IMIE, NAZWISKO, STANOWISKO,
NUMERKONTAKTU, STATUS, STREFA_STREFAID)
VALUES (5, 'Tomasz', 'Nowak', 'Specjalista', 'NUM-4567', 'Aktywny', 5);

------
przeladunek_czas

INSERT INTO przeladunek_czas (przeladunekid, czastrwania)
VALUES (1, 10.5);

INSERT INTO przeladunek_czas (przeladunekid, czastrwania)
```

```
VALUES (2, 12.0);

INSERT INTO przeladunek_czas (przeladunekid, czastrwania)
VALUES (3, 8.25);

INSERT INTO przeladunek_czas (przeladunekid, czastrwania)
VALUES (4, 15.0);

INSERT INTO przeladunek_czas (przeladunekid, czastrwania)
VALUES (5, 9.75);

-----
przesylka

INSERT INTO przesylka (przesylkaid, typprzesylkiid, waga, wymiary, klientid, magazynid,
dataprzyjecia, datawydania, status, ubezpieczenieid, kontener_kontenerid,
przeladunek_przeladunekid)
VALUES (1, 1, 150.50, '120x80x100', 1, 1, TO_DATE('2025-01-14', 'YYYY-MM-DD'),
TO_DATE('2025-01-20', 'YYYY-MM-DD'), 'W trakcie', 1, 1, 1);

INSERT INTO przesylka (przesylkaid, typprzesylkiid, waga, wymiary, klientid, magazynid,
dataprzyjecia, datawydania, status, ubezpieczenieid, kontener_kontenerid,
przeladunek_przeladunekid)
VALUES (2, 2, 200.75, '150x90x120', 2, 2, TO_DATE('2025-01-15', 'YYYY-MM-DD'),
TO_DATE('2025-01-22', 'YYYY-MM-DD'), 'Zakończona', 2, 2, 2);

INSERT INTO przesylka (przesylkaid, typprzesylkiid, waga, wymiary, klientid, magazynid,
dataprzyjecia, datawydania, status, ubezpieczenieid, kontener_kontenerid,
przeladunek_przeladunekid)
VALUES (3, 3, 120.30, '100x60x80', 3, 3, TO_DATE('2025-01-16', 'YYYY-MM-DD'),
TO_DATE('2025-01-23', 'YYYY-MM-DD'), 'Wysłana', 3, 3, 3);

INSERT INTO przesylka (przesylkaid, typprzesylkiid, waga, wymiary, klientid, magazynid,
dataprzyjecia, datawydania, status, ubezpieczenieid, kontener_kontenerid,
przeladunek_przeladunekid)
VALUES (4, 4, 175.00, '140x85x110', 4, 4, TO_DATE('2025-01-17', 'YYYY-MM-DD'),
TO_DATE('2025-01-25', 'YYYY-MM-DD'), 'W trakcie', 4, 4, 4);

INSERT INTO przesylka (przesylkaid, typprzesylkiid, waga, wymiary, klientid, magazynid,
dataprzyjecia, datawydania, status, ubezpieczenieid, kontener_kontenerid,
przeladunek_przeladunekid)
VALUES (5, 5, 250.60, '160x100x130', 5, 5, TO_DATE('2025-01-18', 'YYYY-MM-DD'),
TO_DATE('2025-01-28', 'YYYY-MM-DD'), 'Oczekuje na wysyłkę', 5, 5, 5);
--------
przesylka_kontener

INSERT INTO przesylka_kontener (id, przesylkaid, kontenerid, ilosc, datadodania)
VALUES (1, 1, 1, 10.50, TO_DATE('2025-01-14', 'YYYY-MM-DD'));

INSERT INTO przesylka_kontener (id, przesylkaid, kontenerid, ilosc, datadodania)
VALUES (2, 2, 2, 15.75, TO_DATE('2025-01-15', 'YYYY-MM-DD'));

INSERT INTO przesylka_kontener (id, przesylkaid, kontenerid, ilosc, datadodania)
VALUES (3, 3, 3, 20.00, TO_DATE('2025-01-16', 'YYYY-MM-DD'));

INSERT INTO przesylka_kontener (id, przesylkaid, kontenerid, ilosc, datadodania)
VALUES (4, 4, 4, 25.25, TO_DATE('2025-01-17', 'YYYY-MM-DD'));
```

```
INSERT INTO przesylka_kontener (id, przesylkaid, kontenerid, ilosc, datadodania)
VALUES (5, 5, 5, 30.50, TO_DATE('2025-01-18', 'YYYY-MM-DD'));
-------
ubezpieczenia

INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca)
VALUES (1, TO_DATE('25/07/12', 'DD/MM/YY'), TO_DATE('25/12/18', 'DD/MM/YY'));

INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca)
VALUES (2, TO_DATE('25/07/12', 'DD/MM/YY'), TO_DATE('25/05/26', 'DD/MM/YY'));

INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca)
VALUES (3, TO_DATE('25/01/05', 'DD/MM/YY'), TO_DATE('25/01/05', 'DD/MM/YY'));

INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca)
VALUES (4, TO_DATE('25/05/26', 'DD/MM/YY'), TO_DATE('25/03/25', 'DD/MM/YY'));

INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca)
VALUES (5, TO_DATE('25/05/26', 'DD/MM/YY'), TO_DATE('25/03/25', 'DD/MM/YY'));

INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca)
VALUES (6, TO_DATE('25/06/12', 'DD/MM/YY'), TO_DATE('25/12/18', 'DD/MM/YY'));

INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca)
VALUES (7, TO_DATE('25/01/12', 'DD/MM/YY'), TO_DATE('25/05/26', 'DD/MM/YY'));

INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca)
VALUES (8, TO_DATE('25/11/03
-------
typ_ubezpieczenia

INSERT INTO typ_ubezpieczenia (typubezpieczenia, kwota, ubezpieczenie_ubezpieczenieid)
VALUES ('Typ A', 1000.00, 1);

INSERT INTO typ_ubezpieczenia (typubezpieczenia, kwota, ubezpieczenie_ubezpieczenieid)
VALUES ('Typ B', 1500.50, 2);

INSERT INTO typ_ubezpieczenia (typubezpieczenia, kwota, ubezpieczenie_ubezpieczenieid)
VALUES ('Typ C', 2000.75, 3);

INSERT INTO typ_ubezpieczenia (typubezpieczenia, kwota, ubezpieczenie_ubezpieczenieid)
VALUES ('Typ D', 2500.25, 4);

INSERT INTO typ_ubezpieczenia (typubezpieczenia, kwota, ubezpieczenie_ubezpieczenieid)
VALUES ('Typ E', 3000.80, 5);
--------
warunki ubezpieczenia

INSERT INTO warunki_ubezpieczenia (warunki, typ_ubezpieczenia_typubezpieczenia)
VALUES ('Warunki A', 'Typ A');

INSERT INTO warunki_ubezpieczenia (warunki, typ_ubezpieczenia_typubezpieczenia)
VALUES ('Warunki B', 'Typ B');

INSERT INTO warunki_ubezpieczenia (warunki, typ_ubezpieczenia_typubezpieczenia)
VALUES ('Warunki C', 'Typ C');
```

```
INSERT INTO warunki_ubezpieczenia (warunki, typ_ubezpieczenia_typubezpieczenia)
VALUES ('Warunki D', 'Typ D');

INSERT INTO warunki_ubezpieczenia (warunki, typ_ubezpieczenia_typubezpieczenia)
VALUES ('Warunki E', 'Typ E');
--------
przeladunek
INSERT INTO przeladunek (przeladunekid, statekid, ciezarowkaid, status,
harmonogramprzeladunkow_przeladunekid, przeladunek_czas_przeladunekid)
VALUES (1, 1, 1, 'Aktywny', 1, 1);

INSERT INTO przeladunek (przeladunekid, statekid, ciezarowkaid, status,
harmonogramprzeladunkow_przeladunekid, przeladunek_czas_przeladunekid)
VALUES (2, 2, 2, 'W trakcie', 2, 2);

INSERT INTO przeladunek (przeladunekid, statekid, ciezarowkaid, status,
harmonogramprzeladunkow_przeladunekid, przeladunek_czas_przeladunekid)
VALUES (3, 3, 3, 'Zakończony', 3, 3);

INSERT INTO przeladunek (przeladunekid, statekid, ciezarowkaid, status,
harmonogramprzeladunkow_przeladunekid, przeladunek_czas_przeladunekid)
VALUES (4, 4, 4, 'Wstrzymany', 4, 4);

INSERT INTO przeladunek (przeladunekid, statekid, ciezarowkaid, status,
harmonogramprzeladunkow_przeladunekid, przeladunek_czas_przeladunekid)
VALUES (5, 5, 5, 'Aktywny', 5, 5);
------
przeladunek_przesylka

INSERT INTO przeladunek_przesylka (id, przeladunekid, przesylkaid, zaladowanailosc,
czasoperacji)
VALUES (1, 1, 1, 5000.00, 1.5);

INSERT INTO przeladunek_przesylka (id, przeladunekid, przesylkaid, zaladowanailosc,
czasoperacji)
VALUES (2, 2, 2, 6000.50, 2.0);

INSERT INTO przeladunek_przesylka (id, przeladunekid, przesylkaid, zaladowanailosc,
czasoperacji)
VALUES (3, 3, 3, 7000.75, 2.5);

INSERT INTO przeladunek_przesylka (id, przeladunekid, przesylkaid, zaladowanailosc,
czasoperacji)
VALUES (4, 4, 4, 8000.25, 3.0);

INSERT INTO przeladunek_przesylka (id, przeladunekid, przesylkaid, zaladowanailosc,
czasoperacji)
VALUES (5, 5, 5, 9000.80, 3.5);
------
strefa
INSERT INTO strefa (strefaid, nazwa) VALUES (1, 'Strefa A');
INSERT INTO strefa (strefaid, nazwa) VALUES (2, 'Strefa B');
INSERT INTO strefa (strefaid, nazwa) VALUES (3, 'Strefa C');
INSERT INTO strefa (strefaid, nazwa) VALUES (4, 'Strefa D');
INSERT INTO strefa (strefaid, nazwa) VALUES (5, 'Strefa E');
------
```

pracownik strefa
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (1, 1, 1);
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (2, 2, 2);
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (3, 3, 3);
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (4, 4, 4);
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (5, 5, 5);
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (6, 6, 6);
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (7, 7, 7);
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (8, 8, 8);
-----pracownik
INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid)
VALUES (1, 'Ewa', 'Szymański', 1, 'NUM-5688', 'Inactive', 1);

INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid)
VALUES (2, 'Jan', 'Kamiński', 2, 'NUM-2424', 'Inactive', 2);

INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid)
VALUES (3, 'Marek', 'Szymański', 3, 'NUM-1760', 'Completed', 3);

INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid)
VALUES (4, 'Marek', 'Wiśniewski', 4, 'NUM-4578', 'Completed', 4);

INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid)
VALUES (5, 'Jan', 'Lewandowski', 5, 'NUM-2070', 'Pending', 5);

INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid)
VALUES (6, 'Ewa', 'Zieliński', 6, 'NUM-6680', 'Active', 6);

INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid)
VALUES (7, 'Anna', 'Wójcik', 7, 'NUM-5948', 'Inactive', 7);

INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid)
VALUES (8, 'Anna', 'Wójcik', 8, 'NUM-1206', 'Failed', 8);
---
przesylka
INSERT INTO PRZESYLKA (PRZESYLKAID, TYPPRZESYLKIID, WAGA, WYMIARY, KLIENTID, MAGAZYNID, DATAPRZYJECIA, DATAWYDANIA, STATUS, UBEZPIECZENIEID, KONTENER_KONTENERID, PRZELADUNEK_PRZELADUNEKID)
VALUES (1, 101, 150.50, '120x80x100', 1, 1, TO_DATE('2025-01-14', 'YYYY-MM-DD'), TO_DATE('2025-01-20', 'YYYY-MM-DD'), 'W drodze', 1, 1, 1);

INSERT INTO PRZESYLKA (PRZESYLKAID, TYPPRZESYLKIID, WAGA, WYMIARY, KLIENTID, MAGAZYNID, DATAPRZYJECIA, DATAWYDANIA, STATUS, UBEZPIECZENIEID, KONTENER_KONTENERID, PRZELADUNEK_PRZELADUNEKID)
VALUES (2, 102, 200.00, '150x90x120', 2, 2, TO_DATE('2025-01-15', 'YYYY-MM-DD'), TO_DATE('2025-01-25', 'YYYY-MM-DD'), 'Zakończona', 2, 2, 2);

INSERT INTO PRZESYLKA (PRZESYLKAID, TYPPRZESYLKIID, WAGA, WYMIARY, KLIENTID, MAGAZYNID, DATAPRZYJECIA, DATAWYDANIA, STATUS,

```
UBEZPIECZENIEID, KONTENER_KONTENERID, PRZELADUNEK_PRZELADUNEKID)
VALUES (3, 103, 180.75, '100x60x80', 3, 3, TO_DATE('2025-01-16', 'YYYY-MM-DD'),
TO_DATE('2025-01-22', 'YYYY-MM-DD'), 'Wysłana', 3, 3, 3);

INSERT INTO PRZESYLKA (PRZESYLKAID, TYPPRZESYLKIID, WAGA, WYMIARY,
KLIENTID, MAGAZYNID, DATAPRZYJECIA, DATAWYDANIA, STATUS,
UBEZPIECZENIEID, KONTENER_KONTENERID, PRZELADUNEK_PRZELADUNEKID)
VALUES (4, 104, 225.00, '200x120x150', 4, 4, TO_DATE('2025-01-18', 'YYYY-MM-DD'),
TO_DATE('2025-01-28', 'YYYY-MM-DD'), 'W trakcie', 4, 4, 4);

INSERT INTO PRZESYLKA (PRZESYLKAID, TYPPRZESYLKIID, WAGA, WYMIARY,
KLIENTID, MAGAZYNID, DATAPRZYJECIA, DATAWYDANIA, STATUS,
UBEZPIECZENIEID, KONTENER_KONTENERID, PRZELADUNEK_PRZELADUNEKID)
VALUES (5, 105, 300.00, '180x90x140', 5, 5, TO_DATE('2025-01-19', 'YYYY-MM-DD'),
TO_DATE('2025-01-30', 'YYYY-MM-DD'), 'Oczekuje', 5, 5, 5);

INSERT INTO PRZESYLKA (PRZESYLKAID, TYPPRZESYLKIID, WAGA, WYMIARY,
KLIENTID, MAGAZYNID, DATAPRZYJECIA, DATAWYDANIA, STATUS,
UBEZPIECZENIEID, KONTENER_KONTENERID, PRZELADUNEK_PRZELADUNEKID)
VALUES (6, 106, 500.25, '150x70x100', 1, 1, TO_DATE('2025-02-01', 'YYYY-MM-DD'),
TO_DATE('2025-02-10', 'YYYY-MM-DD'), 'Wysłana', 6, 1, 6);

INSERT INTO PRZESYLKA (PRZESYLKAID, TYPPRZESYLKIID, WAGA, WYMIARY,
KLIENTID, MAGAZYNID, DATAPRZYJECIA, DATAWYDANIA, STATUS,
UBEZPIECZENIEID, KONTENER_KONTENERID, PRZELADUNEK_PRZELADUNEKID)
VALUES (7, 107, 180.00, '120x60x80', 2, 3, TO_DATE('2025-01-25', 'YYYY-MM-DD'),
TO_DATE('2025-02-02', 'YYYY-MM-DD'), 'Zakończona', 7, 3, 7);

INSERT INTO PRZESYLKA (PRZESYLKAID, TYPPRZESYLKIID, WAGA, WYMIARY,
KLIENTID, MAGAZYNID, DATAPRZYJECIA, DATAWYDANIA, STATUS,
UBEZPIECZENIEID, KONTENER_KONTENERID, PRZELADUNEK_PRZELADUNEKID)
VALUES (8, 108, 220.50, '140x80x100', 3, 4, TO_DATE('2025-01-26', 'YYYY-MM-DD'),
TO_DATE('2025-02-06', 'YYYY-MM-DD'), 'W drodze', 8, 4, 8);
```

3.4. Kwerendy

```
SELECT CIEZAROWKAID, NUMER_REJESTRACYJNY, TYP, NOSNOSC, STATUS
FROM CIEZAROWKA
WHERE STATUS = 'Aktywna';

SELECT D.DOKID, D.NAZWA, D.MAKS_NOSNOSC
FROM DOK D
JOIN DOK_STAN DS ON D.DOKID = DS.DOKID
WHERE DS.STAN = 'Otwarte';

SELECT H.DOKID, D.NAZWA, H.DATAZAKONCZENIA
FROM HARMONOGRAMDOKOW H
JOIN DOK D ON H.DOKID = D.DOKID
WHERE H.DATAZAKONCZENIA > TO_DATE('2025-01-25', 'YYYY-MM-DD');

SELECT AVG(WAGA) AS SREDNIA_WAGA
FROM PRZESYLKA;
```

```
SELECT MAGAZYNID, COUNT(*) AS LICZBA_PRZESYLEK
FROM PRZESYLKA
GROUP BY MAGAZYNID;

SELECT DZWIGID, STREFAID, MAKSYMALNA_NOSNOSC, TYP, STATUS
FROM DZWIG
WHERE STREFAID > 103;

SELECT MAX(MAKS_NOSNOSC) AS MAKSYMALNA_NOSNOSC
FROM DOK;

SELECT TYP, SUM(NOSNOSC) AS SUMA_NOSNOSCI
FROM CIEZAROWKA
GROUP BY TYP;


SELECT PR.PRZESYLKAID, PR.WAGA, PR.STATUS, K.NAZWA AS KLIENT,
M.TYPMAGAZYNU AS MAGAZYN
FROM PRZESYLKA PR
JOIN KLIENT K ON PR.KLIENTID = K.KLIENTID
JOIN MAGAZYN M ON PR.MAGAZYNID = M.MAGAZYNID;

SELECT P.PRZESYLKAID, P.WAGA, P.STATUS, K.TYP AS KONTENER,
L.LOKALIZACJAID
FROM PRZESYLKA P
JOIN KONTENER K ON P.KONTENER_KONTENERID = K.KONTENERID
JOIN KONTENER_LOKALIZACJA L ON K.KONTENERID = L.KONTENERID;

SELECT M.MAGAZYNID, M.TYPMAGAZYNU, P.PRZESYLKAID, P.WAGA
FROM MAGAZYN M
JOIN PRZESYLKA P ON M.MAGAZYNID = P.MAGAZYNID
WHERE P.WAGA > 200;

SELECT K.KONTENERID, K.TYP, M.MAGAZYNID, M.TYPMAGAZYNU,
M.ZAJETAPRZESTRZEN
FROM KONTENER K
JOIN MAGAZYN M ON K.KONTENERID = M.MAGAZYNID
WHERE M.ZAJETAPRZESTRZEN > 5000;
```

4. Bibliografia

- Elmasri, R., Navathe, S. B. (2016) „Fundamentals of Database Systems”
- W3Schools „SQL Tutorial W3Schools”
- Christopher, M. (2016) „Logistics & Supply Chain Management”
- Oracle Corporation (2023). „Oracle Database SQL Language Reference”